

# Ingegneria del Software

## Linguaggi di Modellazione

Prof. Paolo Giorgini

A.A. 2026/2027

# Modello

- Rappresentazione di un'entità in modo semplificato
  - astrazione di aspetti o dettagli irrilevanti per lo scopo a cui è destinato il modello - focus sulle informazioni importanti

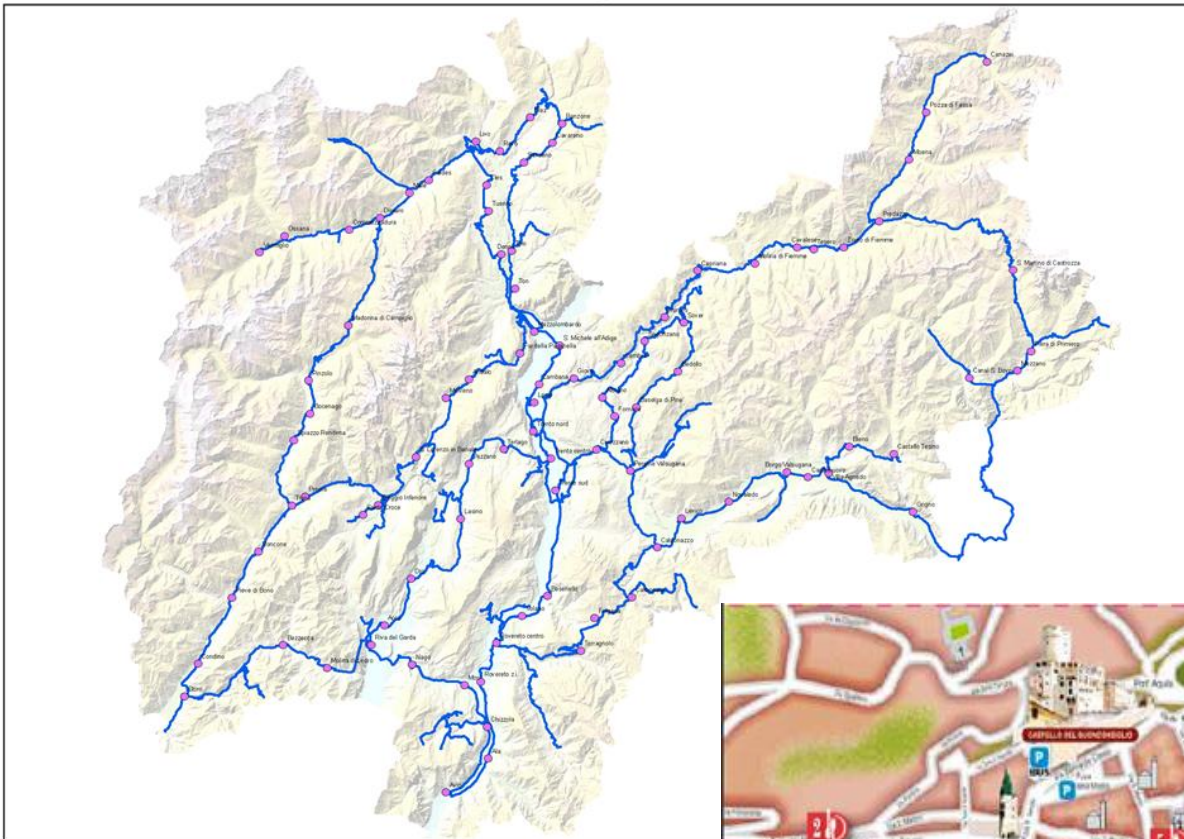


# Notazione del modello



# Quali linguaggi?

- Un ingegnere civile è in grado di leggere i calcoli relativi alla statica di un edificio elaborati da un qualsiasi collega
  - Linguaggio comune della matematica
- Un ingegnere software?
  - Quale linguaggio? UML?
  - Dipende da cosa si intende modellare!
    - Vocabolario di termini che rappresentano i concetti da modellare – utilizzabili solo per alcuni contesti
    - ES. simboli usati per la mappa della città di Trento non sono adatti a modellare una mappa per gli scavi di posa della fibra ottica in Trentino

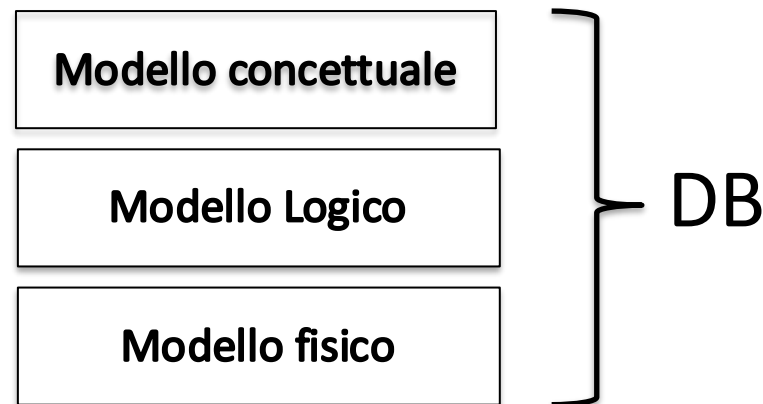


# Scelta del Linguaggio

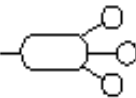
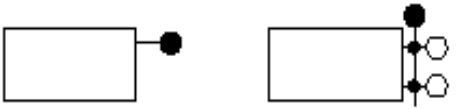
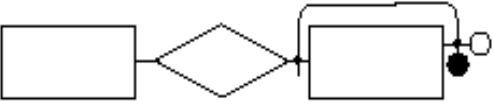
- Facilità d'uso
  - Semplicità di espressione (quanto sia facile produrre un modello) e l'immediatezza della comprensione (quanto sia facile capire il modello)
- Comprensibili anche da non esperti (tecnici)
  - Un'utile strumento di comunicazione con utenti e clienti
- Quanto più formale e preciso è il linguaggio, tanto più il modello può essere utilizzato per analisi ed elaborazioni
  - Formale e preciso non significa necessariamente di difficile da utilizzare
- Linguaggi diversi possono essere utilizzati all'interno delle varie fasi dello sviluppo del software
  - Diversi livelli di astrazione concettuale

# Linguaggio Entity-Relationship

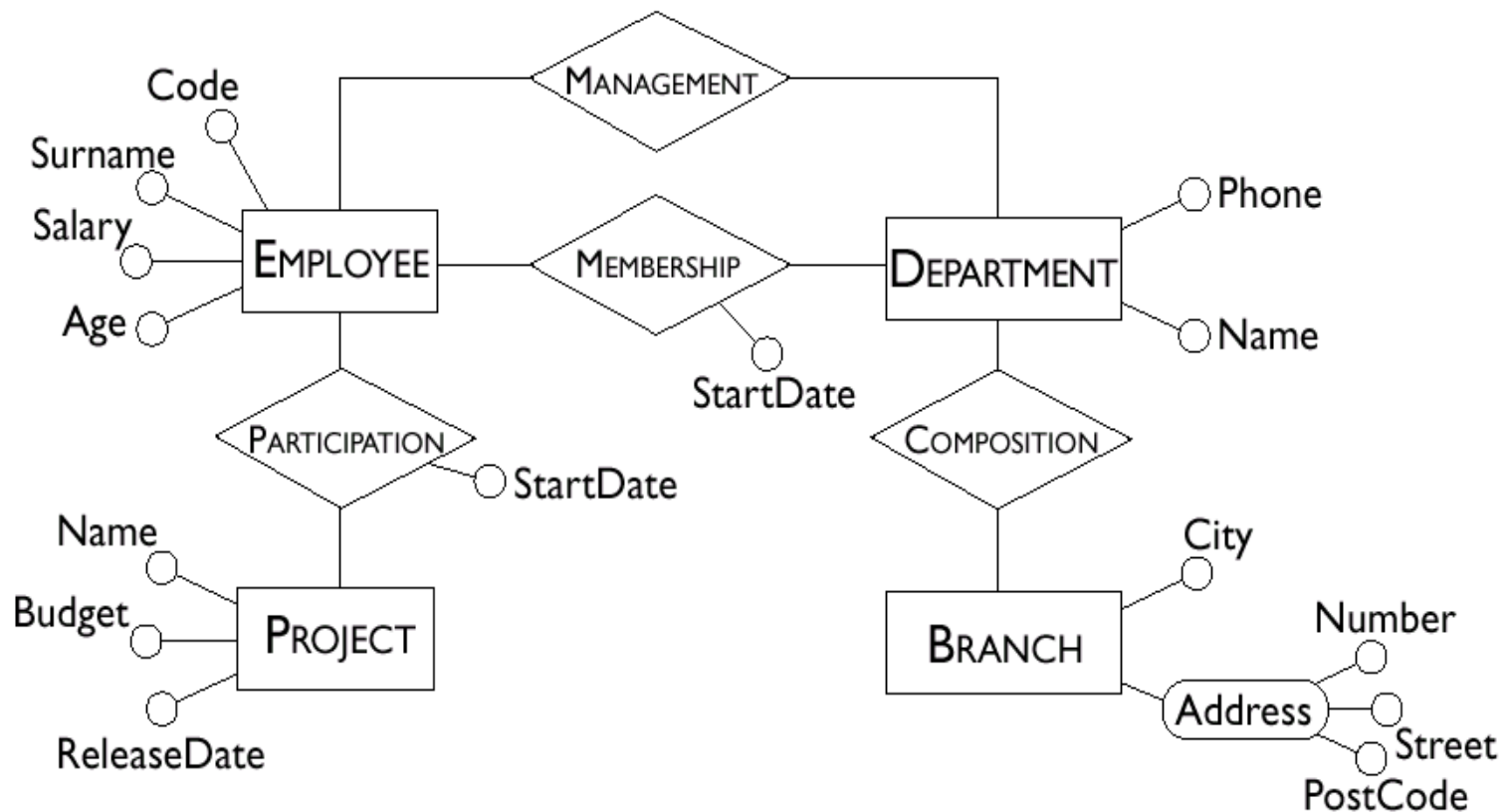
- Utilizzato prevalentemente nell'analisi e progettazione di sistemi informativi e DB
  - Modellazione concettuale (prima di quella logica – modello relazionale)



# Costrutti base

| Construct                   | Graphical representation                                                              |
|-----------------------------|---------------------------------------------------------------------------------------|
| Entity                      |    |
| Relationship                |    |
| Simple attribute            |    |
| Composite attribute         |    |
| Cardinality of a            |    |
| Cardinality of an attribute |    |
| Internal identifier         |    |
| External identifier         |   |
| Generalization              |  |
| Subset                      |  |

# Model with basic attributes



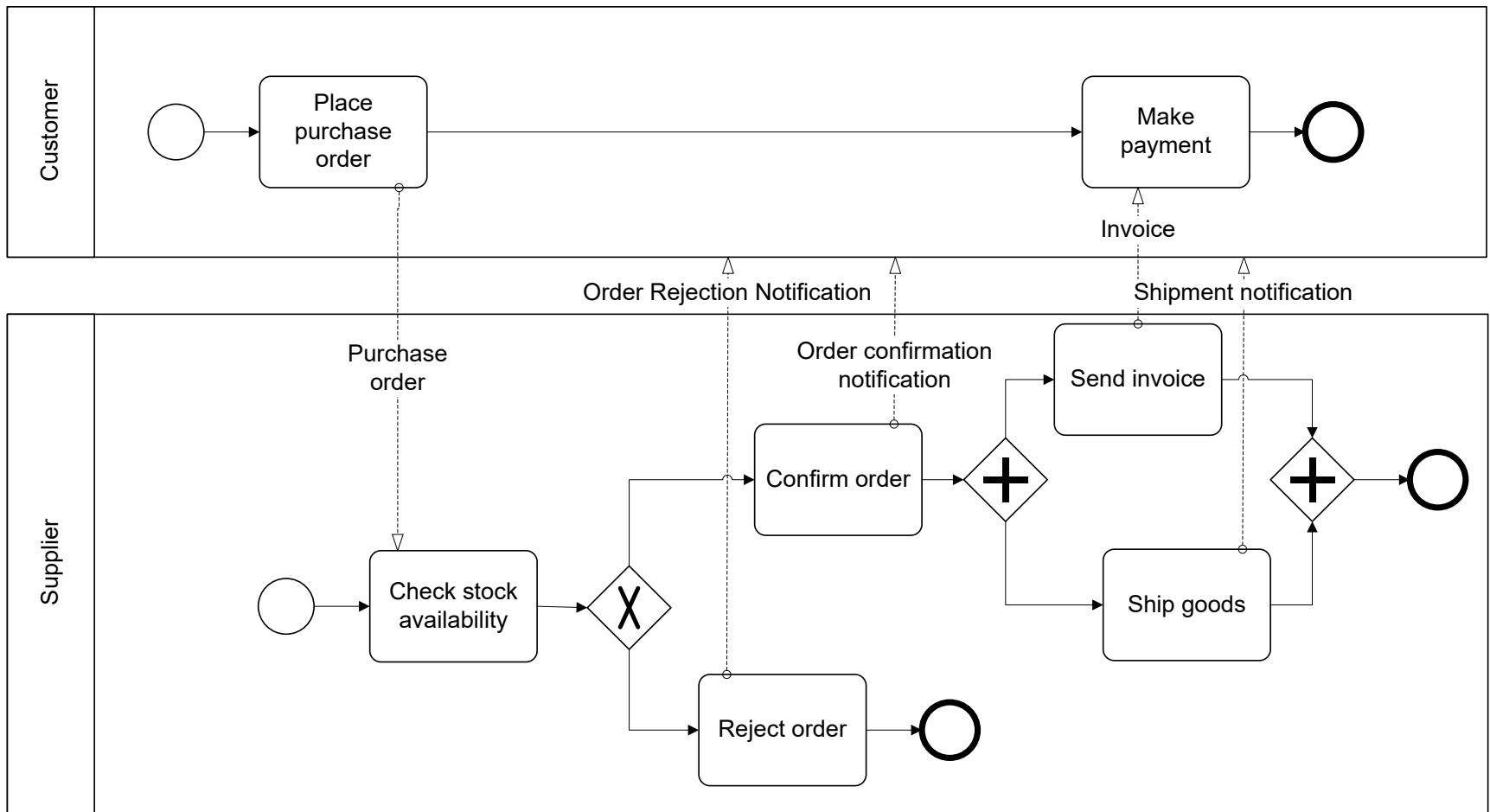
# Se si vuole modellare un processo



# Business Process Modeling Notation

- BPMN is the **OMG standard** for representing business processes
  - Other standards, such as activity diagrams of UML, were not accepted for process modeling in practice, because their use is restricted to the area of object-oriented software design
- There are many tools for designing a BPMN process:
  - BpmnWebModeler (Trisotech)
  - Bizagi Process Modeller (it also provides an execution engine)
  - JBPM (Eclipse plugin)
  - Signavio
  - TIBCO Business Studio (free download, quite large)
  - IBM Websphere Business Modeler
  - ARIS
  - Oracle BPA
  - Business Process Visual Architect (Visual Paradigm)
  - Progress Savvion Business Modeller

# Order Management



## Activities



Task

A Task is a unit of work, the job to be performed. When marked with a  symbol it indicates a Sub-Process, an activity that can be refined.



Transaction

A Transaction is a set of activities that logically belong together; it might follow a specified transaction protocol.



Event Sub-Process

An Event Sub-Process is placed into a Process or Sub-Process. It is activated when its start event gets triggered and can interrupt the higher level process context or run in parallel (non-interrupting) depending on the start event.



Call Activity

A Call Activity is a wrapper for a globally defined Task or Process reused in the current Process. A call to a Process is marked with a  symbol.

### Activity Markers

Markers indicate execution behavior of activities:

-  Sub-Process Marker
-  Loop Marker
-  Parallel MI Marker
-  Sequential MI Marker
-  Ad Hoc Marker
-  Compensation Marker

### Task Types

Types specify the nature of the action to be performed:

-  Send Task
-  Receive Task
-  User Task
-  Manual Task
-  Business Rule Task
-  Service Task
-  Script Task

-  Sequence Flow  
defines the execution order of activities.
-  Default Flow  
is the default branch to be chosen if all other conditions evaluate to false.
-  Conditional Flow  
has a condition assigned that defines whether or not the flow is used.

## Conversations



A Conversation defines a set of logically related message exchanges. When marked with a  symbol it indicates a Sub-Conversation, a compound conversation element.

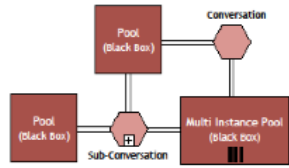


A Call Conversation is a wrapper for a globally defined Conversation or Sub-Conversation. A call to a Sub-conversation is marked with a  symbol.

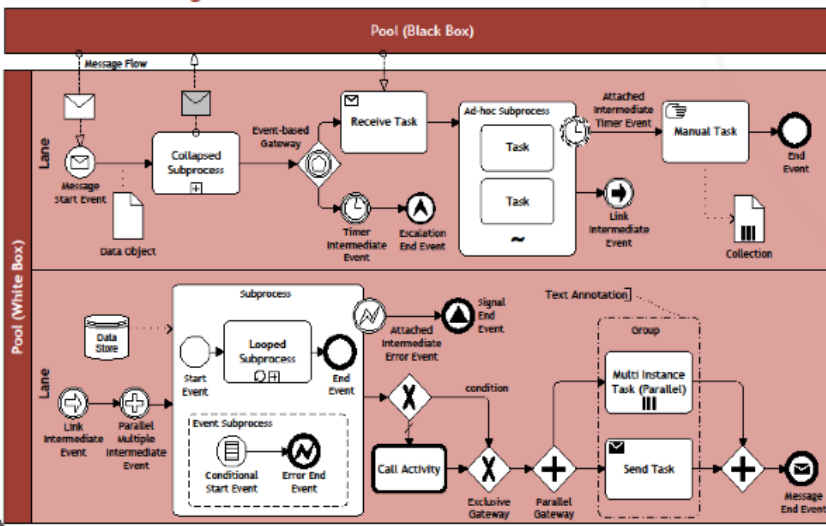


A Conversation Link connects Conversations and Participants.

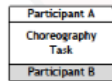
### Conversation Diagram



### Collaboration Diagram



## Choreographies



Participant A

Choreography Task

Participant B

Participant C

Participant D

Participant E

Participant F

Participant G

Participant H

Participant I

Participant J

Participant K

Participant L

Participant M

Participant N

Participant O

Participant P

Participant Q

Participant R

Participant S

Participant T

Participant U

Participant V

Participant W

Participant X

Participant Y

Participant Z

Participant AA

Participant AB

Participant AC

Participant AD

Participant AE

Participant AF

Participant AG

Participant AH

Participant AI

Participant AJ

Participant AK

Participant AL

Participant AM

Participant AN

Participant AO

Participant AP

Participant AQ

Participant AR

Participant AS

Participant AT

Participant AU

Participant AV

Participant AW

Participant AX

Participant AY

Participant AZ

Participant BA

Participant BB

Participant BC

Participant BD

Participant BE

Participant BF

Participant BG

Participant BH

Participant BI

Participant BJ

Participant BK

Participant BL

Participant BM

Participant BN

Participant BO

Participant BP

Participant BQ

Participant BR

Participant BS

Participant BT

Participant BU

Participant BV

Participant BW

Participant BX

Participant BY

Participant BZ

Participant CA

Participant CB

Participant CC

Participant CD

Participant CE

Participant CF

Participant CG

Participant CH

Participant CI

Participant CJ

Participant CK

Participant CL

Participant CM

Participant CN

Participant CO

Participant CP

Participant CQ

Participant CR

Participant CS

Participant CT

Participant CU

Participant CV

Participant CW

Participant CX

Participant CY

Participant CZ

Participant DA

Participant DB

Participant DC

Participant DD

Participant DE

Participant DF

Participant DG

Participant DH

Participant DI

Participant DJ

Participant DK

Participant DL

Participant DM

Participant DN

Participant DO

Participant DP

Participant DQ

Participant DR

Participant DS

Participant DT

Participant DU

Participant DV

Participant DW

Participant DX

Participant DY

Participant DZ

Participant EA

Participant EB

Participant EC

Participant ED

Participant EE

Participant EF

Participant EG

Participant EH

Participant EI

Participant EJ

Participant EK

Participant EL

Participant EM

Participant EN

Participant EO

Participant EP

Participant EQ

Participant ER

Participant ES

Participant ET

Participant EU

Participant EV

Participant EW

Participant EX

Participant EY

Participant EZ

Participant FA

Participant FB

Participant FC

Participant FD

Participant FE

Participant FF

Participant FG

Participant FH

Participant FI

Participant FJ

Participant FK

Participant FL

Participant FM

Participant FN

Participant FO

Participant FP

Participant FQ

Participant FR

Participant FS

Participant FT

Participant FU

Participant FV

Participant FW

Participant FX

Participant FY

Participant FZ

Participant GA

Participant GB

Participant GC

Participant GD

Participant GE

Participant GF

Participant GG

Participant GH

Participant GI

Participant GJ

Participant GK

Participant GL

Participant GM

Participant GN

Participant GO

Participant GP

Participant GQ

Participant GR

Participant GS

Participant GT

Participant GU

Participant GV

Participant GW

Participant GX

Participant GY

Participant GZ

Participant HA

Participant HB

Participant HC

Participant HD

Participant HE

Participant HF

Participant HG

Participant HH

Participant HI

Participant HJ

Participant HK

Participant HL

Participant HM

Participant HN

Participant HO

Participant HP

Participant HQ

Participant HR

Participant HS

Participant HT

Participant HU

Participant HV

Participant HW

Participant HX

Participant HY

Participant HZ

Participant IA

Participant IB

Participant IC

Participant ID

Participant IE

Participant IF

Participant IG

Participant IH

Participant II

Participant IJ

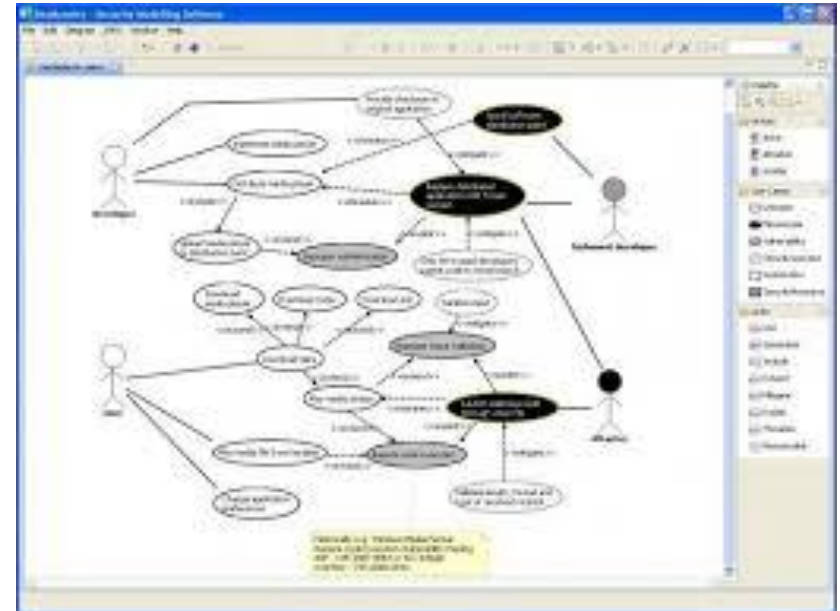
Participant IK

# Sicurezza?



# Linguaggi per la sicurezza

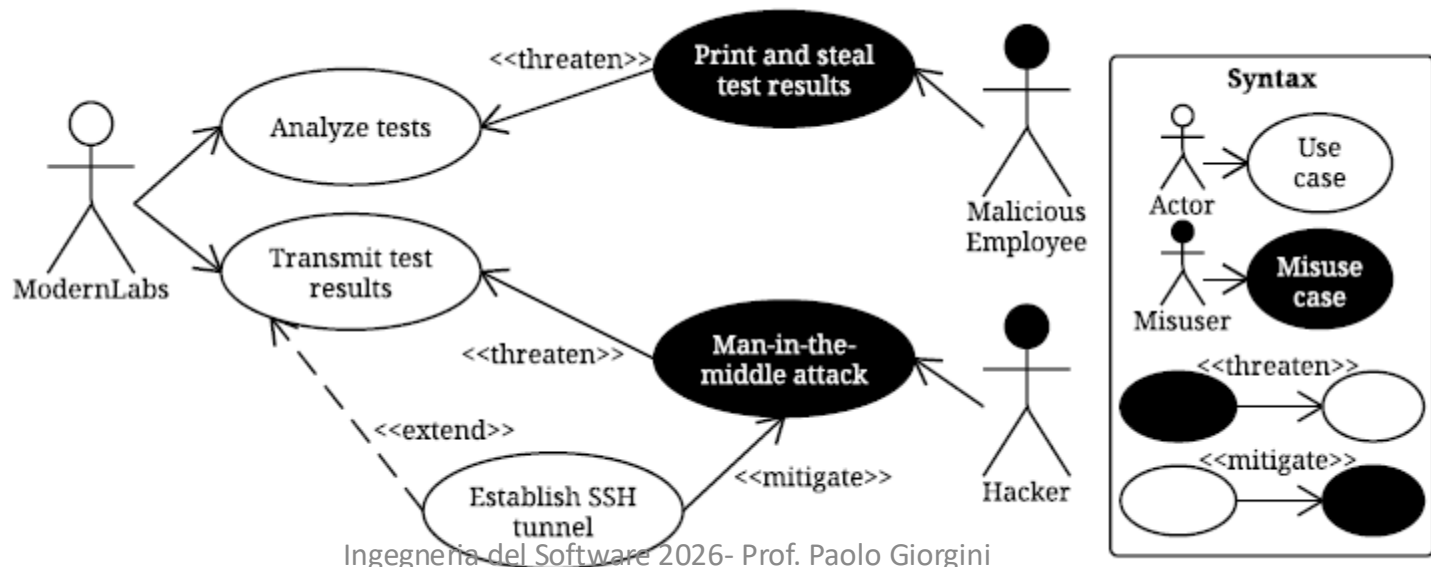
- Molti approcci alla Security Requirements Engineering (SRE) usano linguaggi per costruire modelli del sistema
  - documentazione precisa e rigorosa
  - forme di analisi automatica
- Diversi tipi di modelli
  - UML-based
  - Goal-oriented
  - Machine-oriented
  - Business processes
  - Socio-technical systems



# Linguaggi UML-based

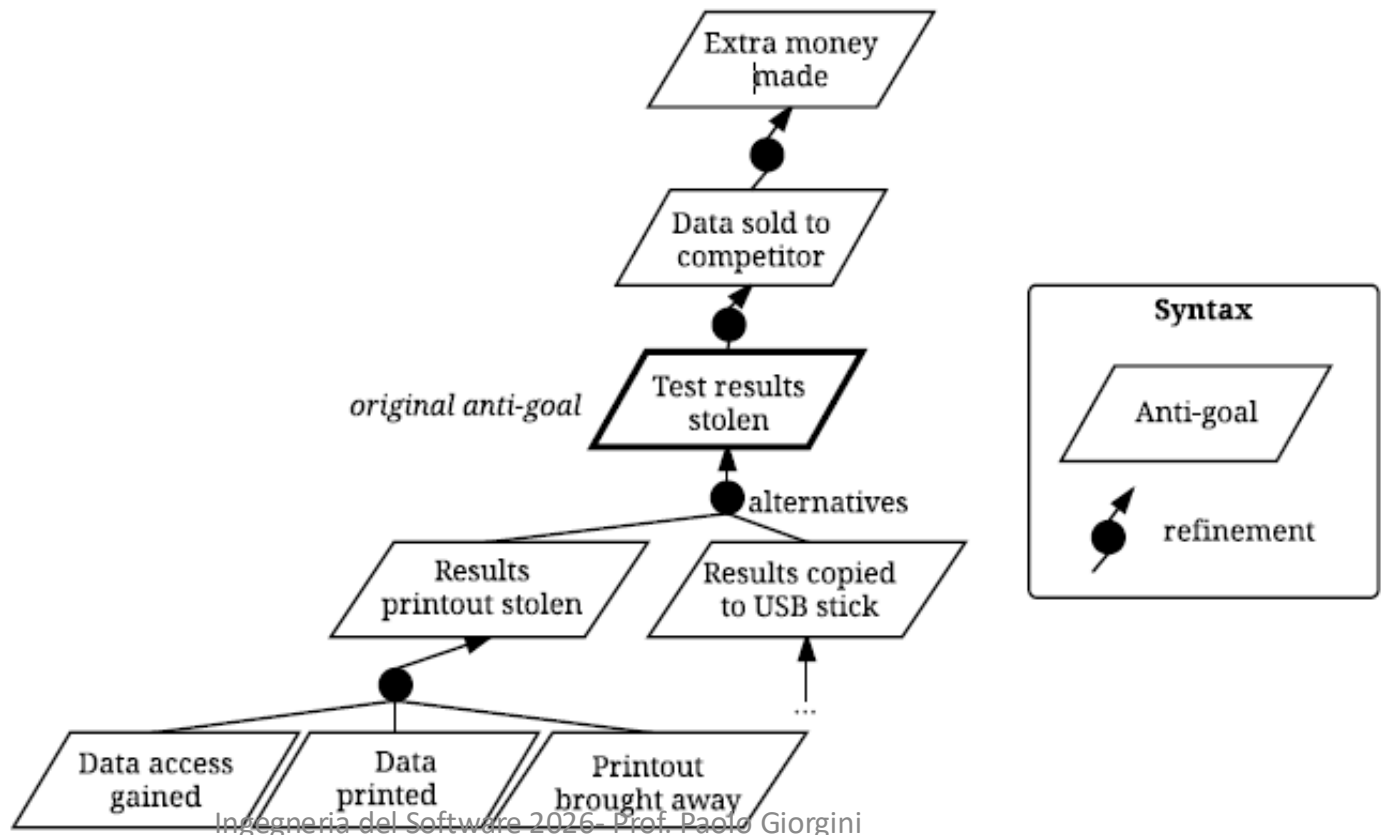
- **Misuse case**

- Alcuni attori sono “malicious” e tentano di sfruttare il sistema
- Altri attori progettano soluzioni di sicurezza per evitare il problema

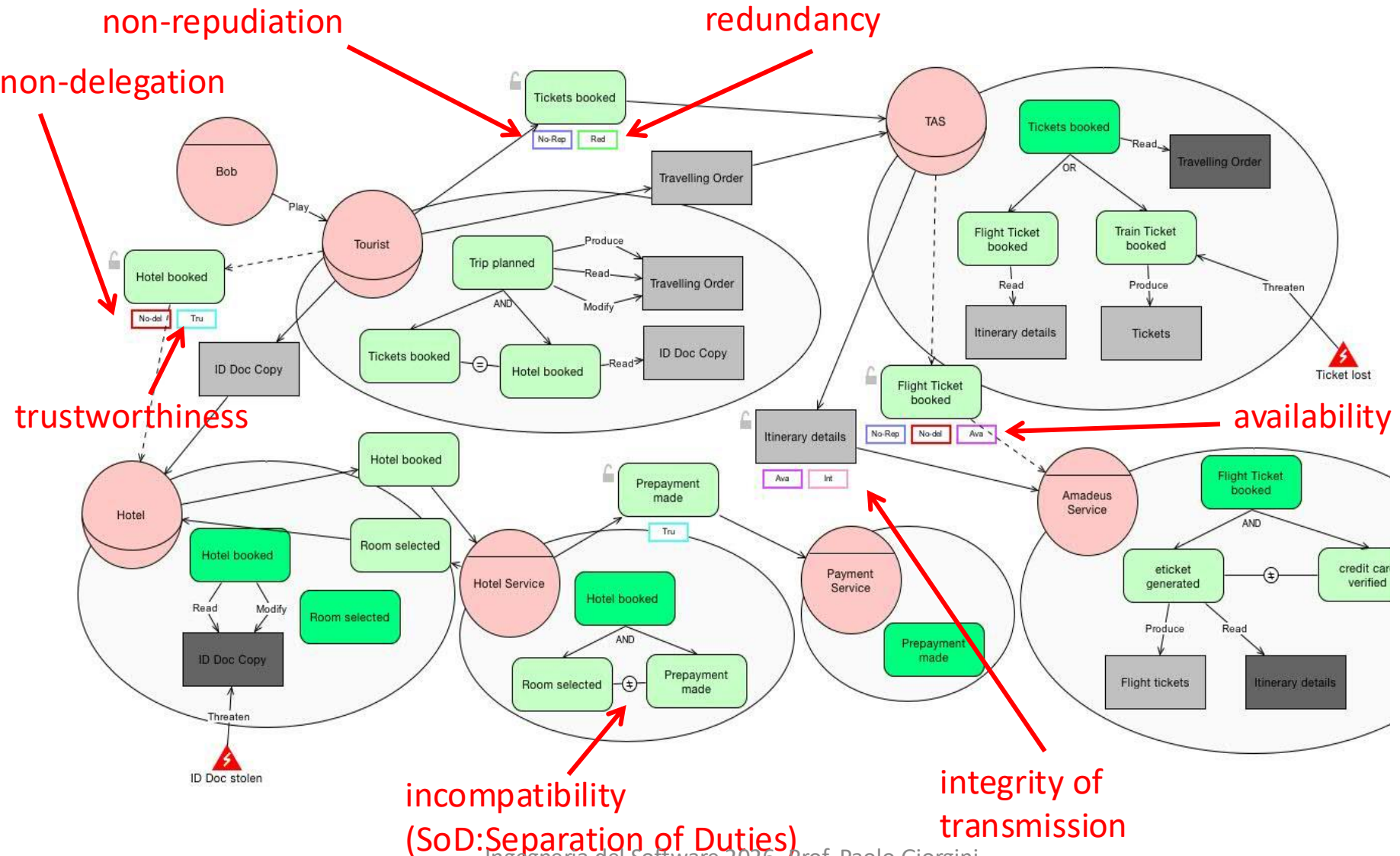


# Linguaggi goals-based

- **Anti-goals:** quali sono gli obiettivi degli "attacher"



# Social view: expressing security needs



# UML

- UML = **U**nified **M**odeling **L**anguage
- Famiglia di notazioni grafiche per il supporto all'analisi e alla progettazione di un sistema software
  - In particolare sistemi OO, ma non solo
- OMG standars
  - Object management group
  - Consorzio di aziende IT
  - No profit
  - Creare degli standard nel contesto IT



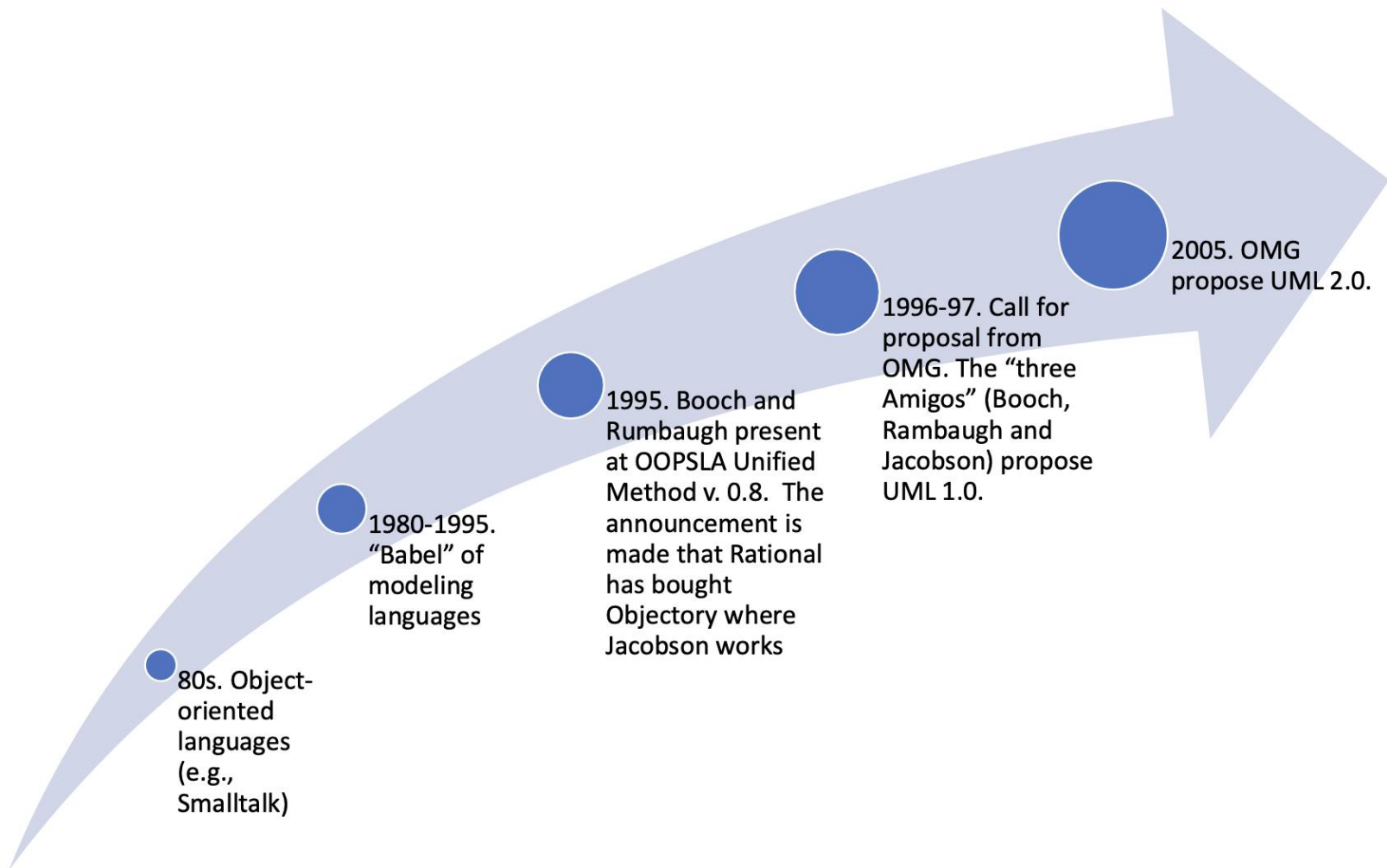
# Che cosa non è UML

- Non è un metodo di sviluppo (metodologia)!
- E' un linguaggio / notazione
- Non è un linguaggio di programmazione
- E' un linguaggio di modellazione
- Non è legato ad una metodologia particolare
- Può essere usato dai metodi più classici fino a quelli agile

# Come usare UML

- Sketch
  - Scopo: facilitare la comunicazione e la discussione - solo le informazioni più importanti (diagrammi incompleti)
  - Tool: editor grafico o lavagna
- Progetto dettagliato (blueprint)
  - Scopo: fornire un modello completo/dettagliato da implementare, in modo che la programmazione sia principalmente un'attività meccanica
  - Tools: modellatore UML
- Verso il linguaggio di programmazione
  - Scopo: fornire un “modello eseguibile”
  - Tools: specifici per eseguire/trasformare il modello

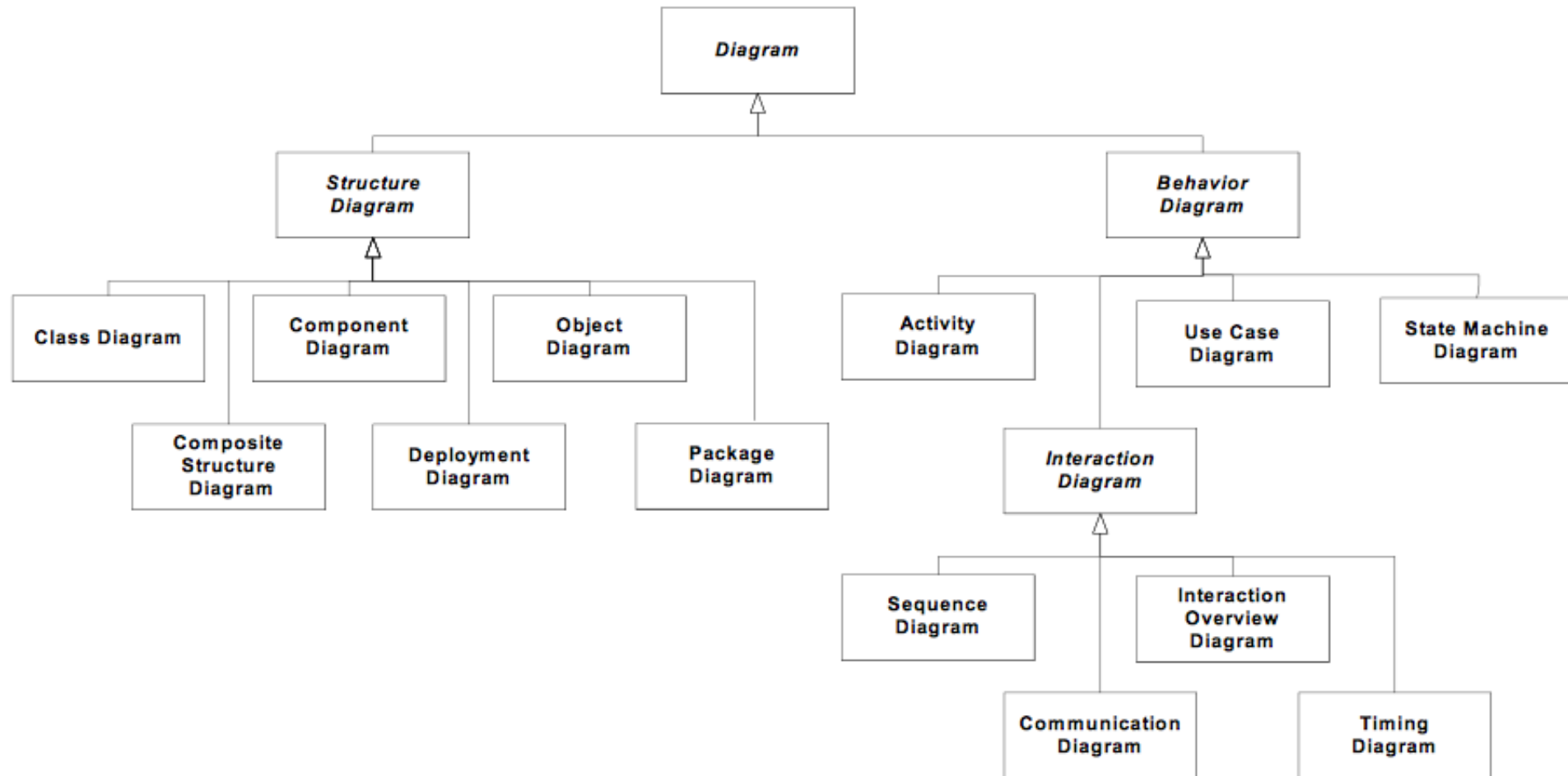
# Un pò di storia



# Perché studiare UML

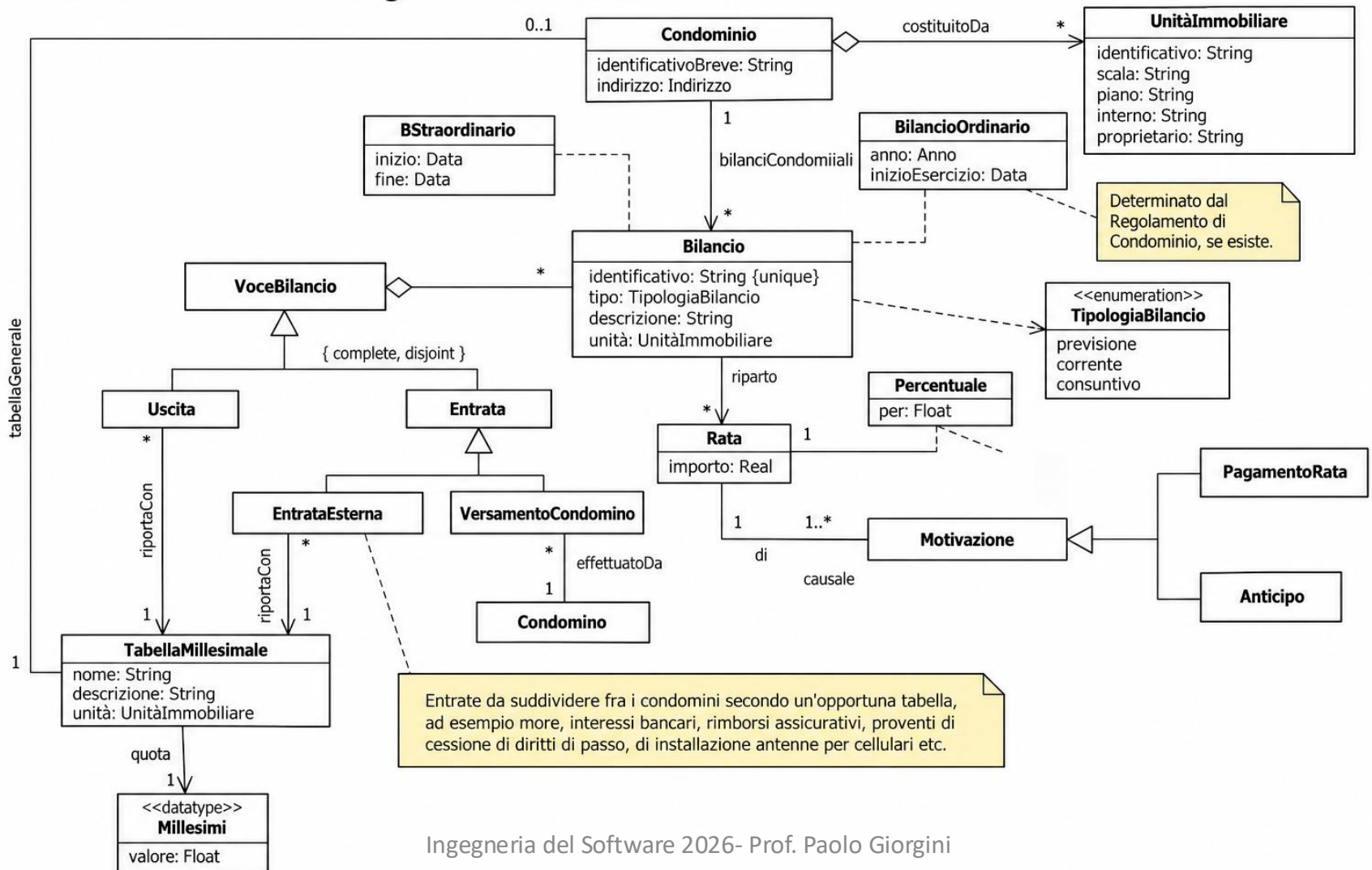
- **Uno Standard Diffuso**
  - è uno standard riconosciuto a livello globale
  - Svolge il ruolo di "lingua franca" nel mondo dell'ICT, facilitando la comunicazione tra team e stakeholders
- **Strumento di Comunicazione Visiva**
  - Un diagramma UML può spiegare concetti complessi in modo molto più efficace rispetto a una descrizione verbale o scritta
  - "Un'immagine vale più di mille parole" - UML aiuta a visualizzare e chiarire i requisiti e le architetture
- **Richiesto dall'Industria**
  - Molto usato a livello industriale, soprattutto per la fase di schizzo dei progetti
  - Richiesto in alcuni bandi pubblici e contratti industriali
- **Supporto alla Progettazione Dettagliata**
  - UML fornisce un modello completo che rende la fase di implementazione più facile e riduce gli errori nel codice

# Costrutti base di UML

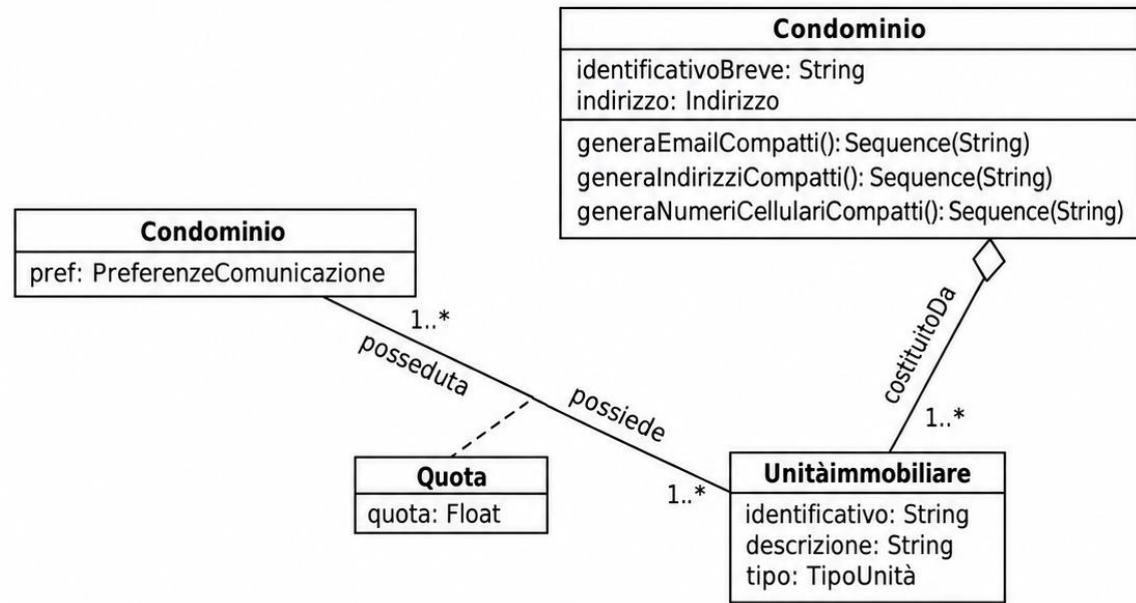


# Class diagram - vari tipi di associazioni

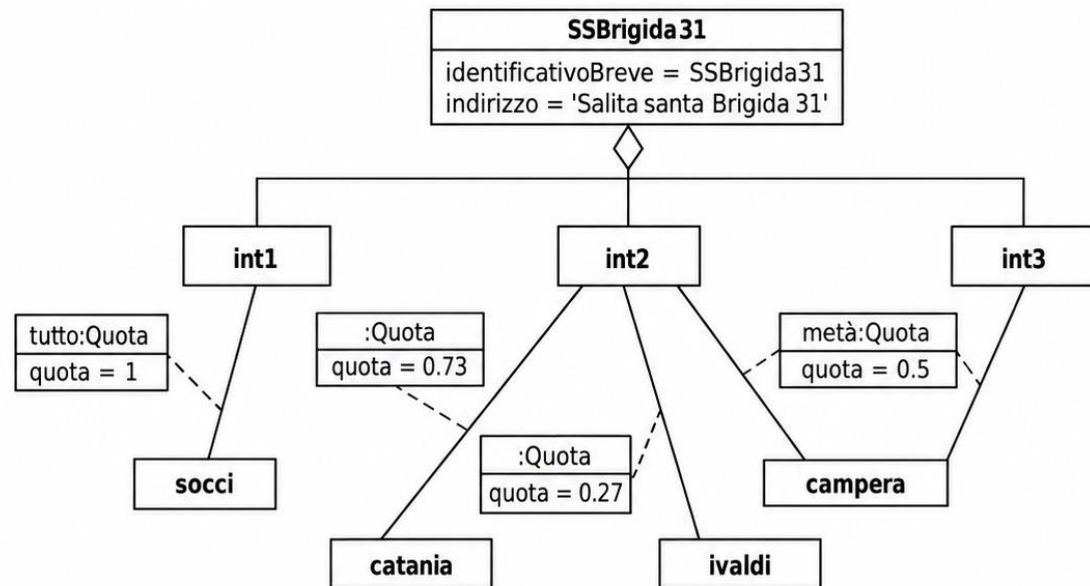
## Gestione Condominio – Diagramma delle Classi



# Class diagram e object diagram



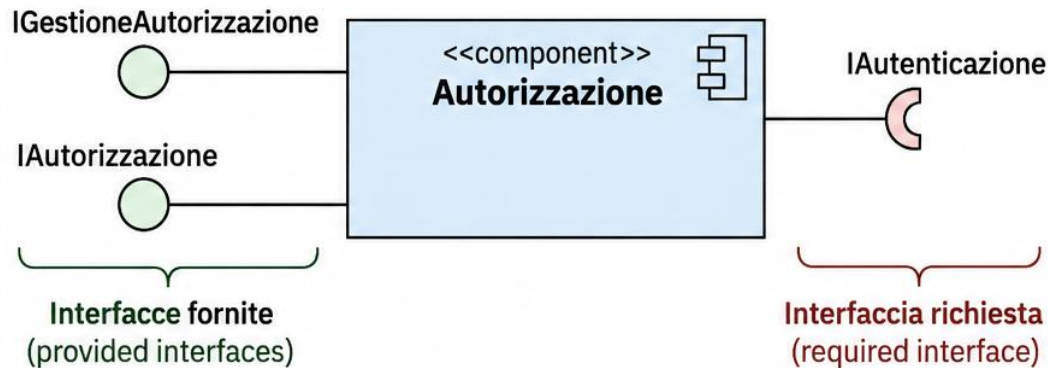
(a)



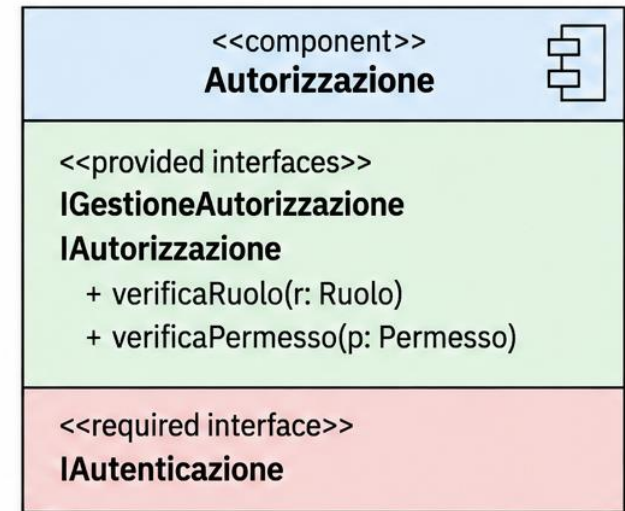
(b)

# Component diagram

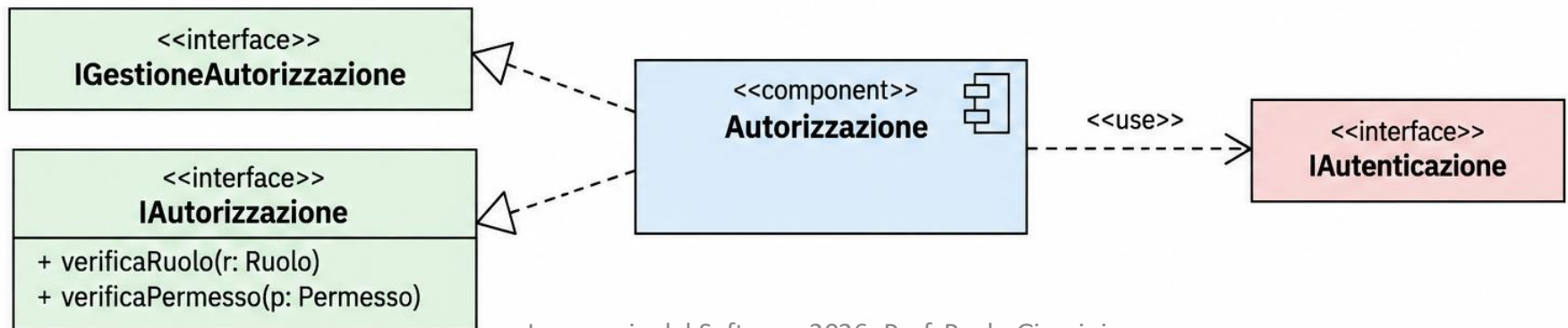
(a) Rappresentazione compatta



(b) Dettaglio delle interfacce nel componente

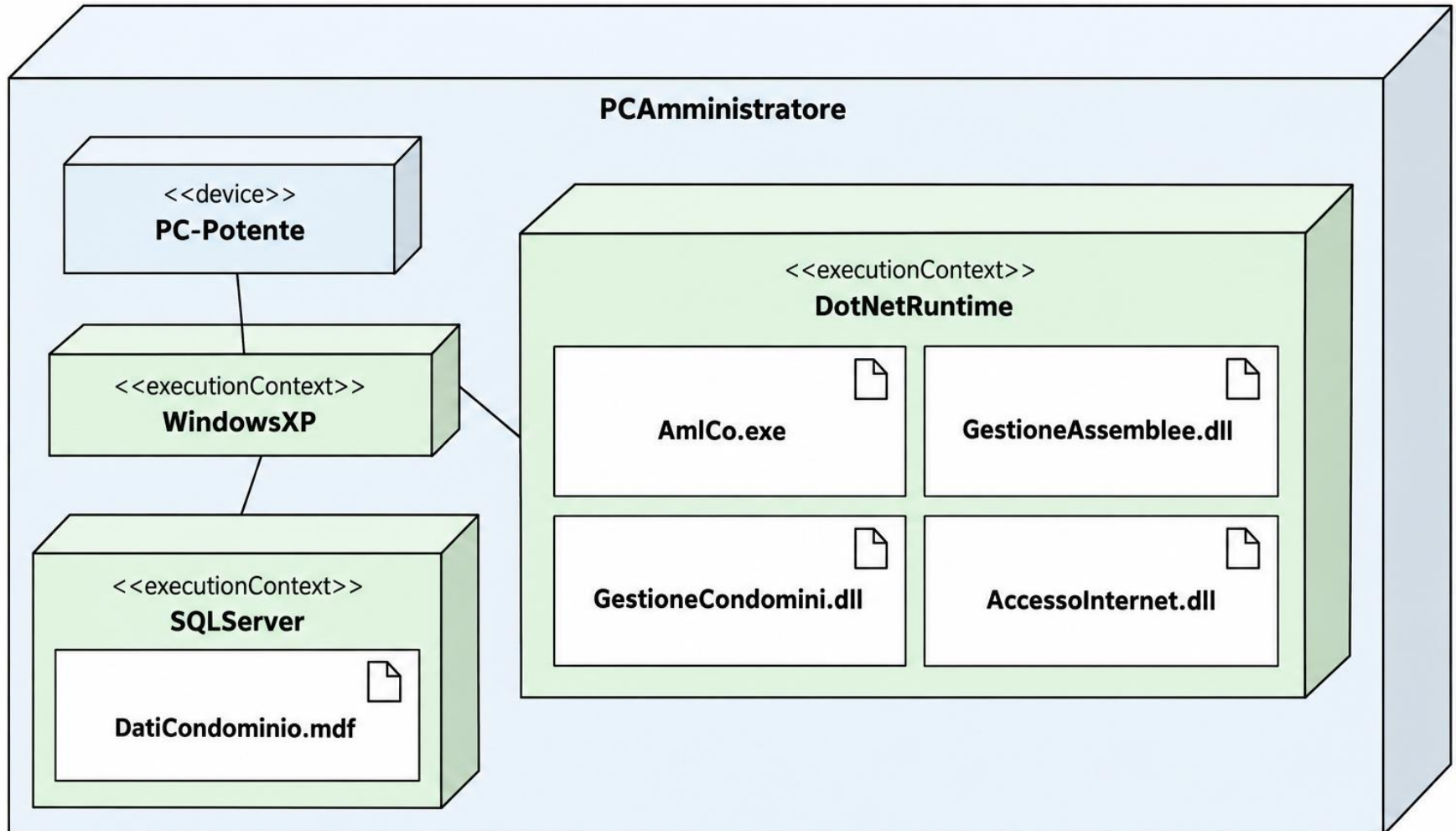


(c) Rappresentazione con interfacce esplicite



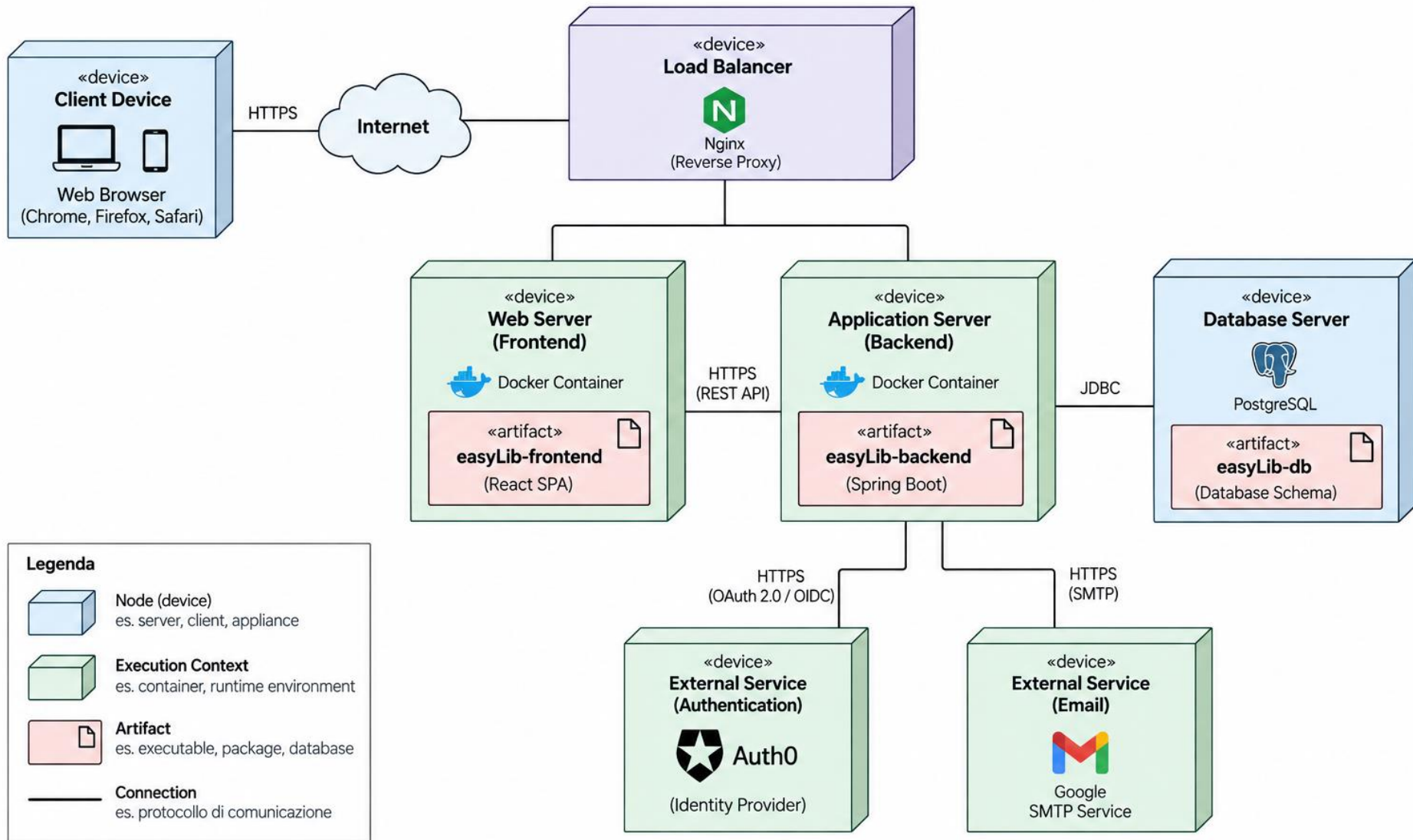
# Deployment diagram

dove vengono eseguiti e installati concretamente gli artefatti software

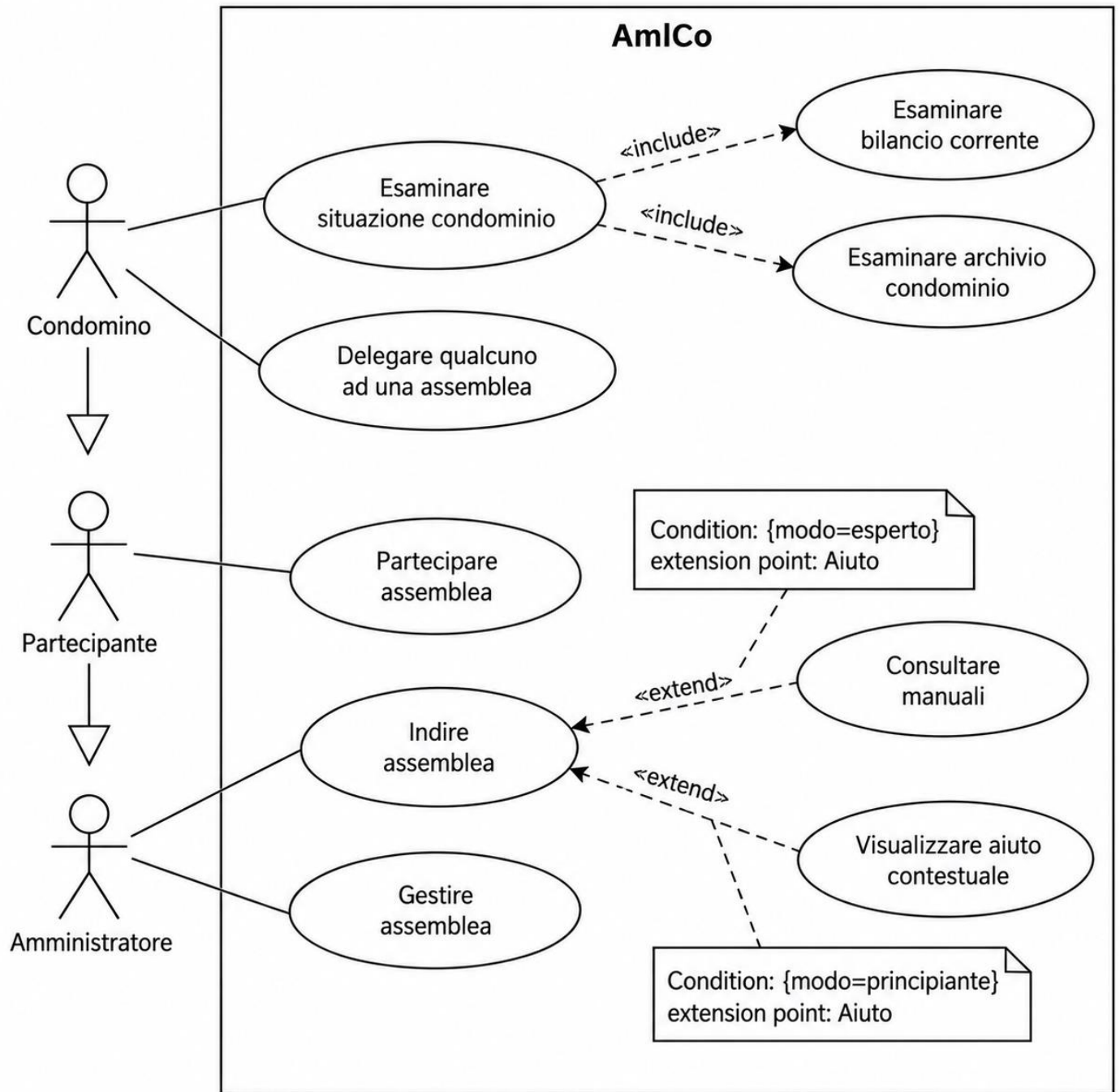


# Esempio di Deployment Diagram – EasyLib (tecnologie moderne)

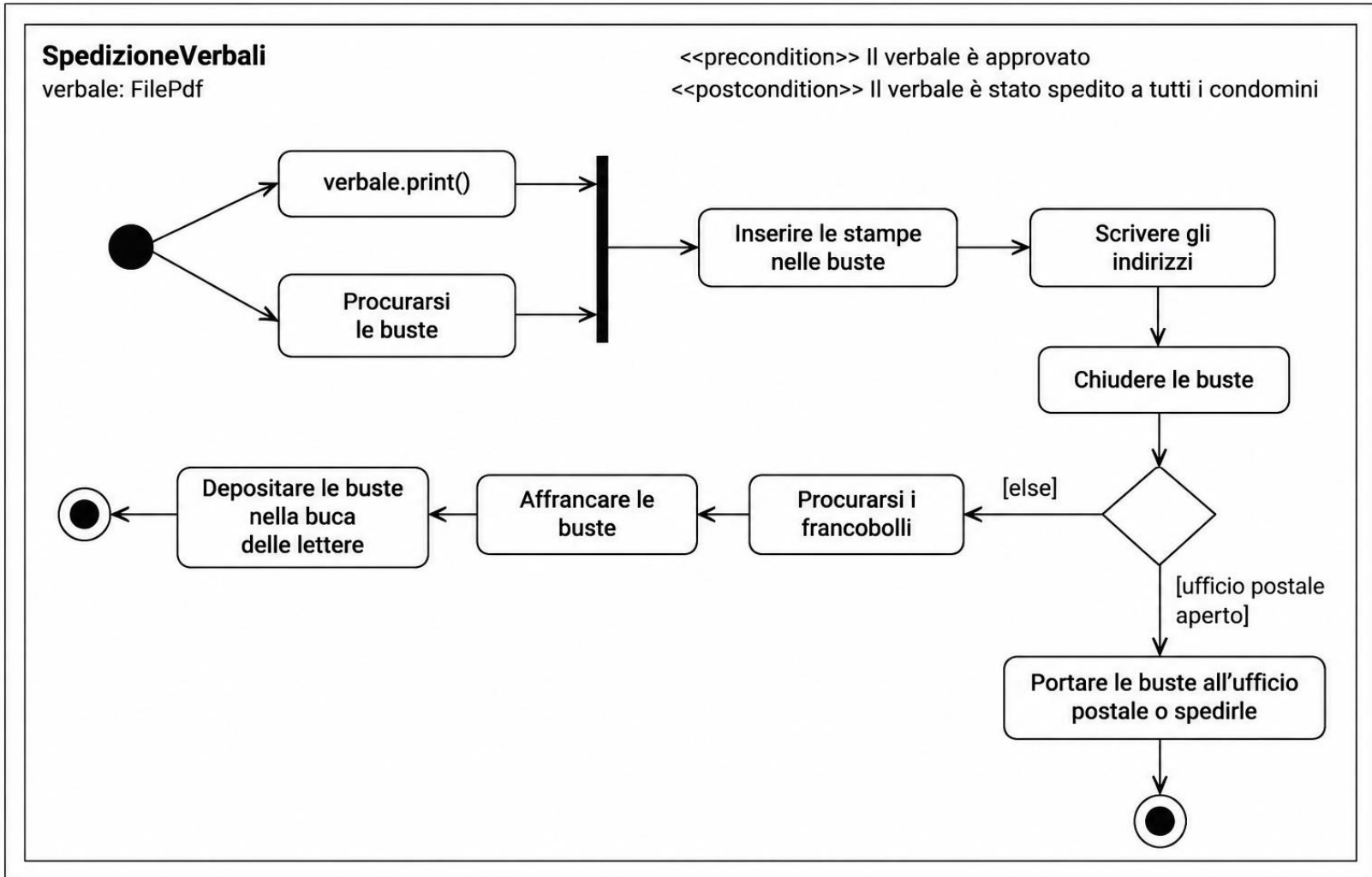
Mostra dove vengono eseguiti i componenti del sistema e quali artefatti sono distribuiti su ciascun nodo.



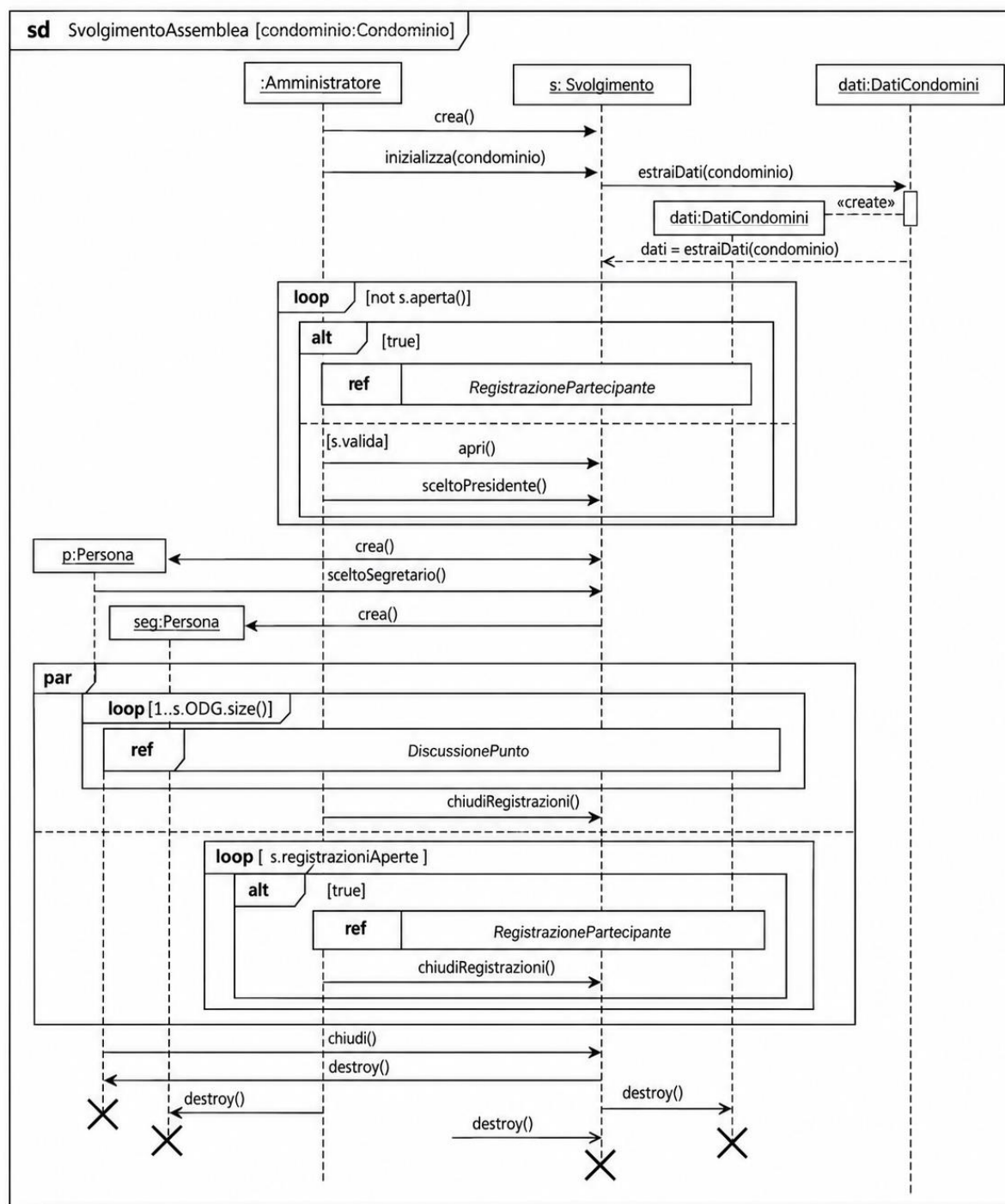
# Use case diagram



# Activity diagram



# Sequence diagram



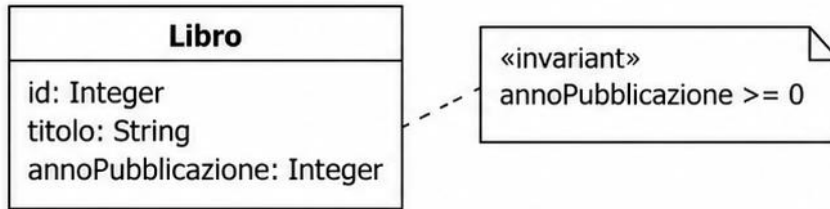
# Il linguaggio OCL

- Object Constraint Language
  - Linguaggio puramente funzionale per scrivere espressioni e vincoli sui modelli object-oriented
  - Proposto come estensione di UML (OMG)
- Linguaggio di query
  - Valutazione di espressioni (valore)
  - Una espressione è una specifica o riferimento ad un valore
  - Un vincolo è una restrizione su uno o più valori o parti di un modello orientato agli oggetti.
    - Es. nel caso di tipo booleano, un'espressione può essere utilizzata per esprimere un constraint, richiedendo che il suo valore sia sempre vero
    - Utilizzo di espressioni OCL come note nel diagramma (contesto)

# OCL: esempio di utilizzo

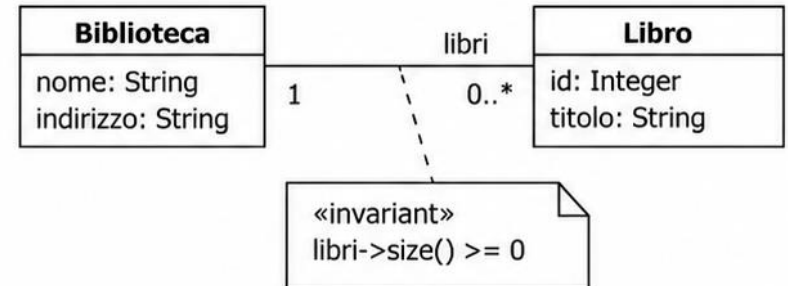
OCL viene utilizzato come vincolo (constraint) nei modelli UML, tipicamente all'interno di note, per esprimere regole precise sul modello.

## 1. Vincolo su un'attributo (classe)



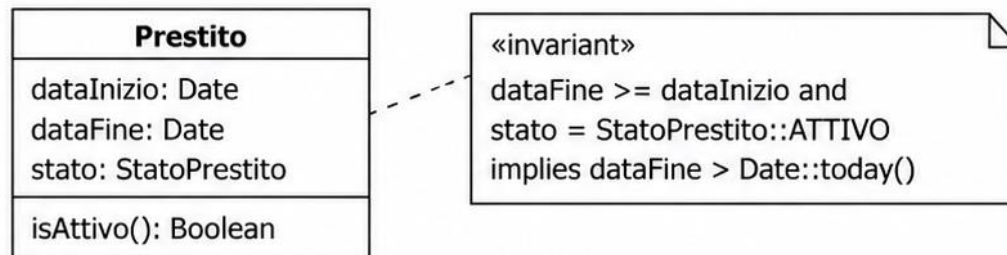
- **Contesto:** classe Libro
- **Significato:** l'anno di pubblicazione deve essere maggiore o uguale a 0.
- L'espressione OCL è di tipo booleano e deve essere sempre vera per ogni istanza di Libro.

## 2. Vincolo su una associazione (molteplicità)



- **Contesto:** associazione Biblioteca–Libro
- **Significato:** una biblioteca può avere zero o più libri.
- Esempio di utilizzo di un'operazione di collezione (`->size()`).

## 3. Vincolo più complesso



- **Contesto:** classe Prestito
- **Significato:** la data di fine deve essere successiva o uguale alla data di inizio e, se il prestito è attivo, la data di fine deve essere nel futuro.
- Mostra l'utilizzo di operatori logici, enumerazioni e operazioni di tipo.

OCL permette di esprimere in modo preciso e non ambiguo le regole del dominio, integrandosi con i diagrammi UML.

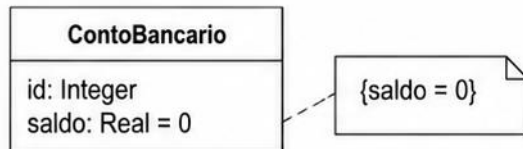
# OCL: tipi di espressioni

- Le espressioni possono essere usate in diversi diagrammi UML:
  - per specificare un **valore iniziale** di un attributo o associazione
  - specificare **valori derivati** di un attributo o associazione
  - per specificare il **corpo di un'operazione**
  - per indicare un'**istanza in un diagramma dinamico**
  - per indicare una **condizione in un diagramma dinamico**
  - per indicare il **valore attuale di un parametro** in un diagramma dinamico

# OCL: esempi di espressioni nei diagrammi UML

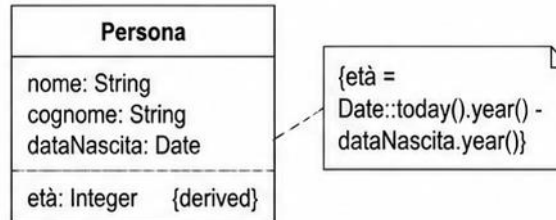
Alcuni esempi di utilizzo di espressioni OCL nei diversi diagrammi UML.

## 1. Valore iniziale di un attributo (Class Diagram)



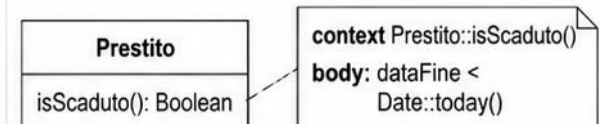
**Significato:** il saldo iniziale di un nuovo conto è 0.

## 2. Valore derivato (Class Diagram)



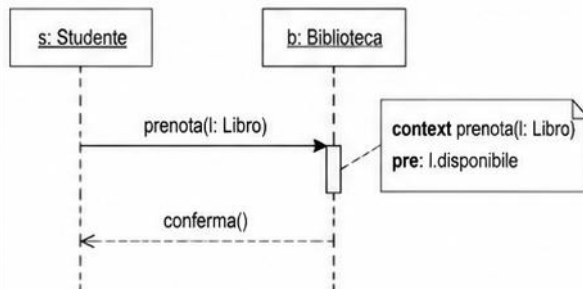
**Significato:** l'età è un valore derivato calcolato a partire dalla data di nascita.

## 3. Corpo di un'operazione (Class Diagram)



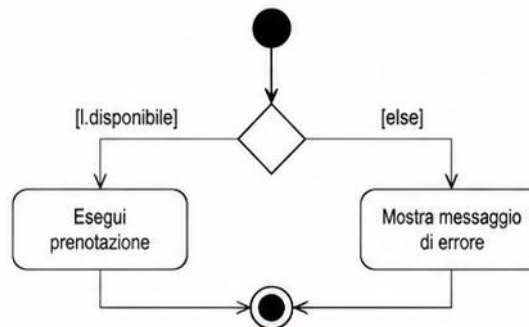
**Significato:** l'operazione restituisce true se la data di fine è precedente alla data odierna.

## 4. Istanza in un diagramma dinamico (Sequence Diagram)



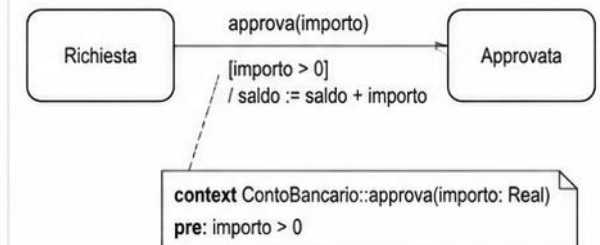
**Significato:** l'istanza 's' (studente) invia un messaggio con parametro 'l' (libro). L'espressione OCL usa l'istanza nella precondizione.

## 5. Condizione in un diagramma dinamico (Activity Diagram)



**Significato:** la guardia della decisione è un'espressione OCL che verifica la disponibilità del libro.

## 6. Valore attuale di un parametro (State Machine Diagram)



**Significato:** la transizione è abilitata solo se il valore attuale del parametro 'importo' è maggiore di 0.

**OCL permette di esprimere in modo preciso regole, calcoli e condizioni, integrandosi con i diversi diagrammi UML.**

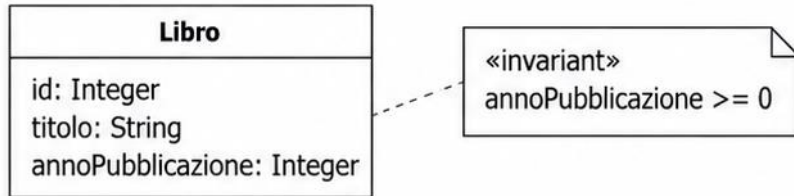
# OCL: Tipi di vincoli

- Vi sono quattro tipi principali di vincoli:
  - Un **invariante** è un vincolo che stabilisce una condizione che deve essere sempre valida per tutte le istanze di una classe o interfaccia. Un'invariante è descritta usando una espressione logica che valutata è sempre vera
  - una **precondizione** per una operazione è una restrizione che deve essere vera prima dell'esecuzione dell'operazione
  - una **postcondizione** per una operazione è una restrizione che deve essere vera subito dopo l'esecuzione dell'operazione
  - una **guardia** è un vincolo che deve essere vero in una fase della vita del sistema. Tipi di guardie sono `init`: e `derive`:

# OCL: esempi di tipi di vincoli

## 1. Invariante

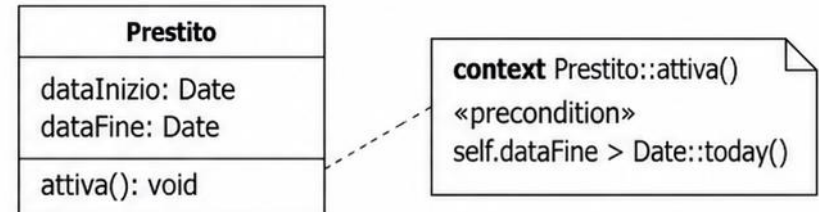
Vincolo che deve essere sempre vero per tutte le istanze.



**Significato:** l'anno di pubblicazione di un libro deve essere maggiore o uguale a 0 per ogni istanza di *Libro*.

## 2. Precondizione

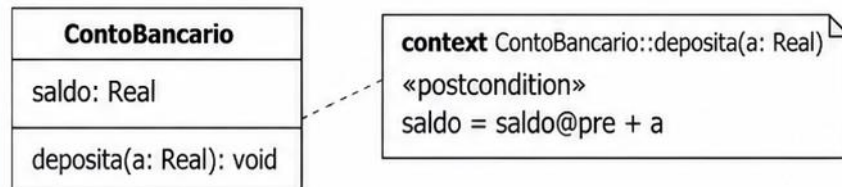
Vincolo che deve essere vero prima dell'esecuzione di un'operazione.



**Significato:** un prestito può essere attivato solo se la data di fine è nel futuro.

## 3. Postcondizione

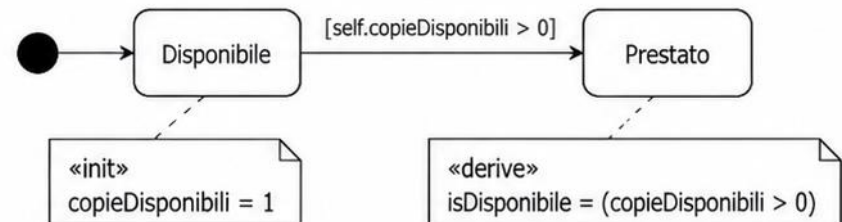
Vincolo che deve essere vero subito dopo l'esecuzione di un'operazione.



**Significato:** dopo il deposito, il saldo deve essere uguale al saldo precedente più l'importo depositato.

## 4. Guardia (init e derive)

Vincolo che deve essere vero in una fase del sistema.



**Significato:** la transizione da Disponibile a Prestato è abilitata solo se ci sono copie disponibili. L'attributo `copieDisponibili` è inizializzato a 1 e l'attributo `isDisponibile` è un valore derivato.

**Gli invarianti, le precondizioni, le postcondizioni e le guardie permettono di specificare in modo preciso le regole del sistema utilizzando espressioni OCL.**

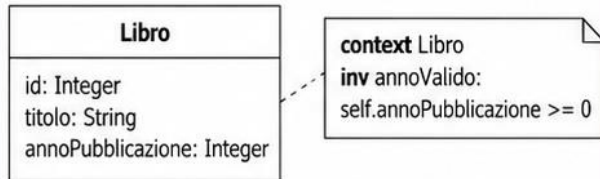
# OCL: contesto di un'espressione

- Specifica la parte del modello su cui agisce l'espressione
  - generalmente è una classe, una interfaccia, un tipo di dato, un componente
- Molte volte la parte interessata del modello è una operazione o un attributo, raramente un'istanza
  - Il contesto indica sempre l'elemento sintattico del modello generalmente definito in UML. Questo elemento è chiamato contesto dell'espressione.
- Successivamente è importante il tipo di contesto dell'espressione. L'espressione viene valutata sempre sugli oggetti (istanze) per cui spesso viene indicato esplicitamente l'istanza mediante la parola chiave **self**.

# OCL: esempi di contesto di un'espressione

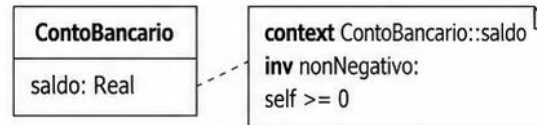
Il contesto indica su quale elemento del modello è definita l'espressione OCL.

## 1. Contesto: classe



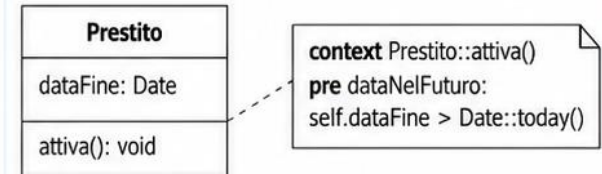
- **Contesto:** classe Libro
- L'espressione riguarda tutte le istanze di Libro.
- self indica l'istanza corrente di Libro.

## 2. Contesto: attributo



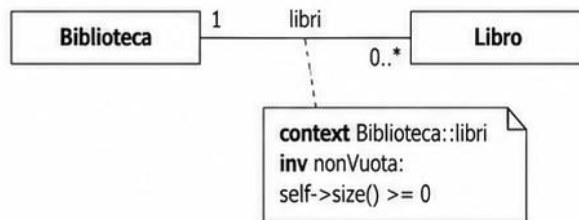
- **Contesto:** attributo saldo della classe ContoBancario.
- L'espressione è valutata sul valore dell'attributo (self è il valore dell'attributo).

## 3. Contesto: operazione



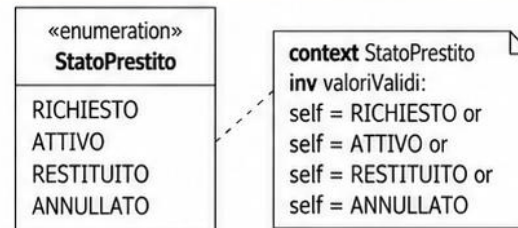
- **Contesto:** operazione attiva() della classe Prestito.
- L'espressione specifica una condizione che deve essere vera prima dell'esecuzione dell'operazione.

## 4. Contesto: associazione



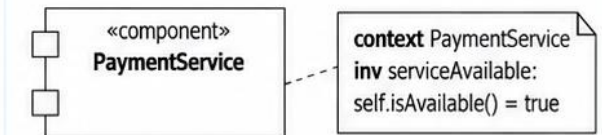
- **Contesto:** associazione libri della classe Biblioteca.
- L'espressione è valutata sulla collezione di libri associati a una biblioteca.

## 5. Contesto: tipo di dato



- **Contesto:** tipo di dato (enumerazione) StatoPrestito.
- L'espressione specifica i valori ammessi.

## 6. Contesto: componente (es. sistema esterno)

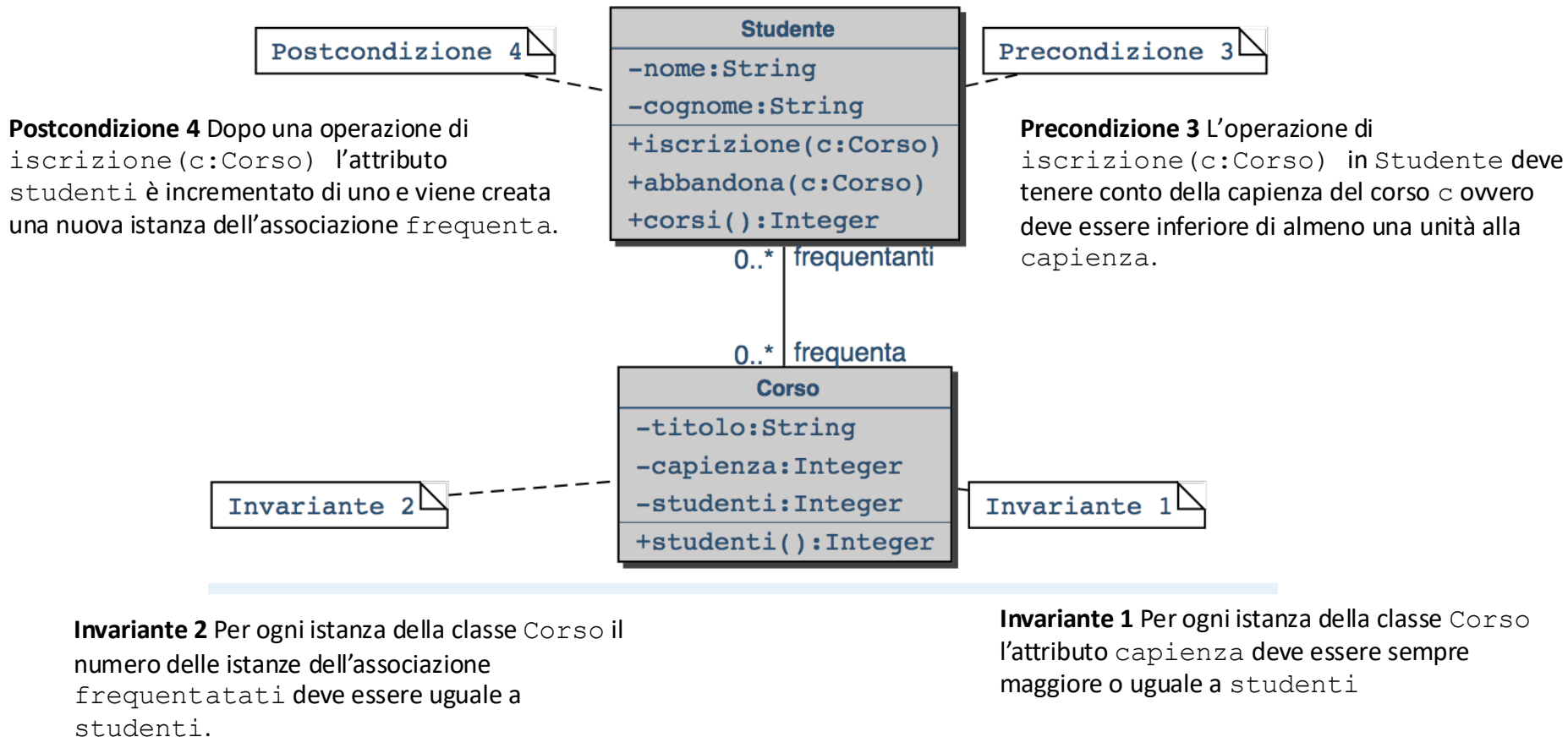


- **Contesto:** componente PaymentService.
- L'espressione specifica un vincolo sullo stato del componente.

**Il contesto permette di capire a quali elementi del modello si riferisce l'espressione OCL e come viene valutata (tipicamente sull'istanza corrente, indicata da self).**

# OCL: esempio (1)

## Modello degli studenti che frequentano corsi



# OCL: esempio (2)

## -- Invariante 1

```
context Corso
inv: capienza >= frequentanti.size()
```

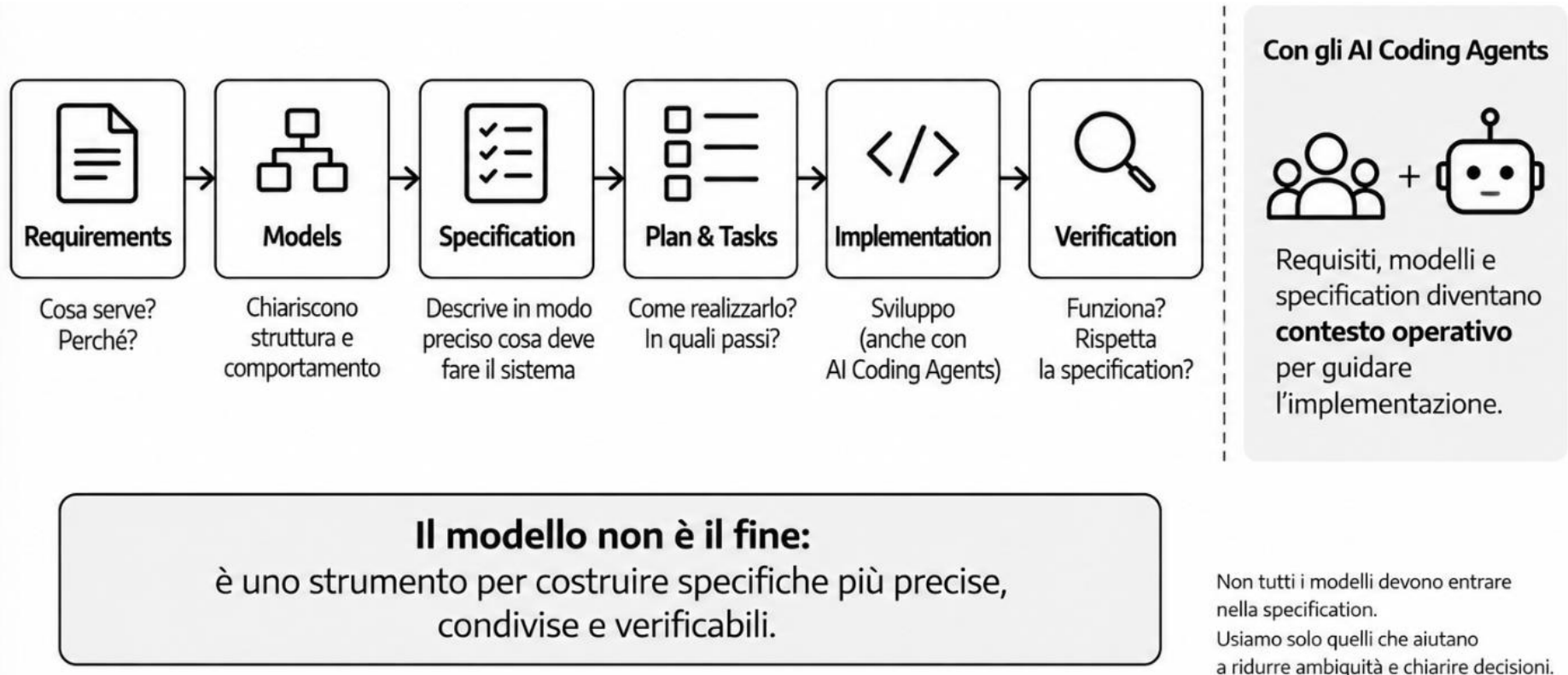
## -- Invariante 2

```
context Corso
inv: studenti = frequentanti.size()
```

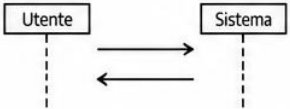
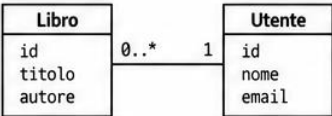
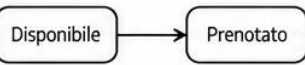
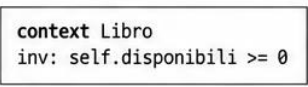
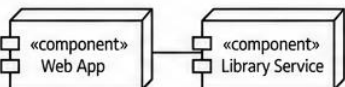
## -- Pre e post condizione 3 e 4

```
context Studente::iscrizione(c:Corso)
pre: c.capienza > c.studenti
post: c.studenti = c.studenti@pre+1
      and c.frequentanti = c.frequentanti@pre+1
```

# Dalla modellazione alla specification



# Quale modello per quale informazione?

| Se voglio chiarire...                             | Posso usare...                 | Esempio di notazione                                                                                        |
|---------------------------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| Chi usa il sistema e per quale obiettivo?         | Use Case Diagram               |  Use Case                 |
| Qual è il flusso di un'attività?                  | Activity Diagram / BPMN        |  Activity / BPMN          |
| Come interagiscono componenti/attori?             | Sequence Diagram               |  Sequence                 |
| Quali sono i concetti e le relazioni del dominio? | Class / Domain Model           |  Class Diagram            |
| Quali stati può assumere un'entità?               | State Machine Diagram          |  State Machine            |
| Quali vincoli devono essere sempre rispettati?    | OCL / Business Rules           |  OCL                     |
| Come è strutturato il sistema?                    | Component / Deployment Diagram |  Component / Deployment |

## Specification

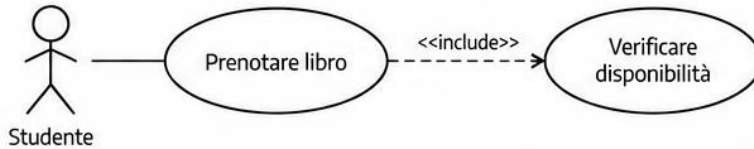
-  Testo e descrizione
-  Scenari e flussi
-  Regole e vincoli
-  Modelli (diagrammi)
-  Acceptance Criteria
-  Altri dettagli (es. NFR, design decision)

**Non serve modellare tutto.**

Modelliamo ciò che è necessario per eliminare ambiguità nella specification.

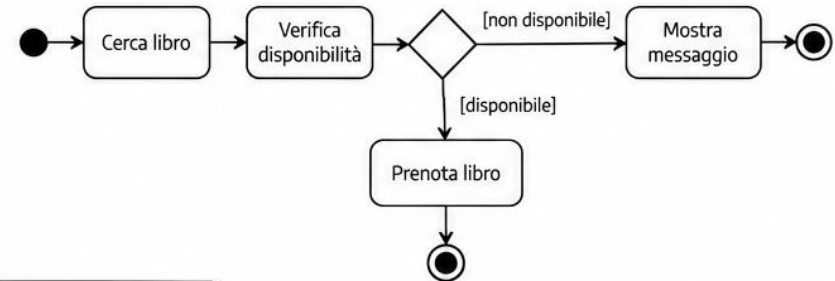
## Use Case

Chi interagisce con il sistema?  
Qual è l'obiettivo?



## Activity Diagram

Qual è il flusso?

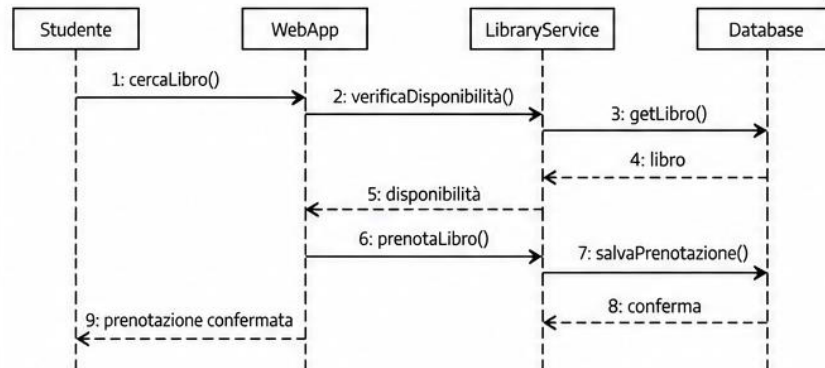


## Feature: Prenotare un libro

Uno studente cerca un libro,  
verifica la disponibilità e, se disponibile,  
lo prenota.

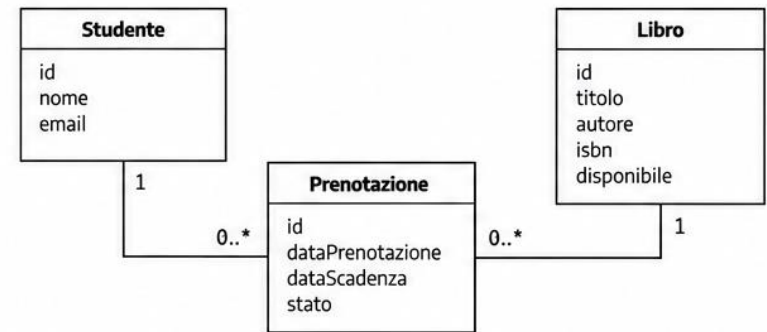
## Sequence Diagram

Chi comunica con chi e in quale ordine?



## Domain / Class Model

Quali concetti sono coinvolti?

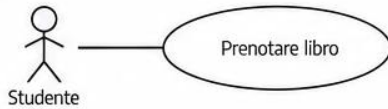


**Ogni modello descrive una prospettiva diversa della stessa feature.**

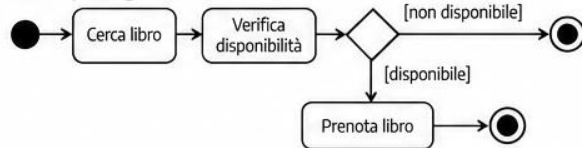
La combinazione dei modelli può ridurre ambiguità che un singolo diagramma non riesce a eliminare.

## Modelli

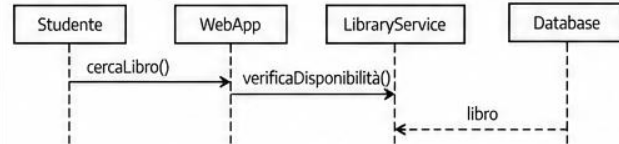
### Use Case



### Activity Diagram



### Sequence Diagram



### Domain Model



## Informazioni estratte



### Attori e obiettivi

- Chi usa il sistema?
- Qual è l'obiettivo?



### Flussi principali e alternativi

- Quali sono le attività?
- Quali sono i casi alternativi?



### Interazioni

- Chi comunica con chi?
- In quale ordine?



### Concetti del dominio

- Quali entità sono coinvolte?
- Come sono correlate?



### Regole e vincoli

- Quali condizioni devono sempre essere rispettate?
- Quali sono i vincoli di business?

## Specification



### Feature description

Descrizione sintetica della funzionalità e del suo obiettivo.

### Scenarios

Scenario principale e scenari alternativi.

### Business Rules

Regole di business derivanti dai modelli.

### Acceptance Criteria

Criteri verificabili per validare la funzionalità.

### Edge Cases

Casi limite e situazioni particolari.

### NFR rilevanti

Requisiti non funzionali correlati (es. performance, sicurezza, usabilità).

## Esempio EasyLib

**Feature:**  
Prenotare un libro



### Scenario principale

Lo studente cerca un libro disponibile e lo prenota.



### Business rule

Un libro non disponibile non può essere prenotato.

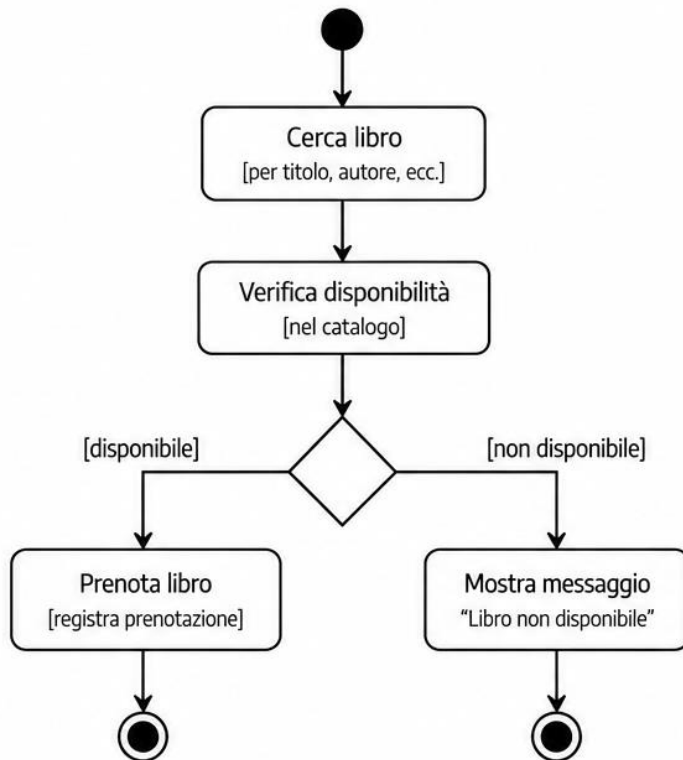


### Acceptance criterion

Dato un libro disponibile, quando lo studente seleziona "Prenota", allora il sistema registra la prenotazione e ne mostra la conferma.

**Il valore del modello sta nelle informazioni che rende esplicite  
e che possono essere riutilizzate nella specification.**

## Activity Diagram (modello)



## Scenario (informazione estratta)

### Scenario principale

1. Lo studente cerca un libro nel catalogo.
2. Il sistema verifica la disponibilità.
3. Se il libro è disponibile, lo studente lo prenota.
4. Il sistema registra la prenotazione e mostra una conferma.



## Business Rule (informazione estratta)

Un libro non disponibile non può essere prenotato.



## Acceptance Criteria (criterio verificabile)

### AC1 – Prenotazione con libro disponibile

**Dato** un libro disponibile nel catalogo,  
**quando** lo studente seleziona "Prenota",  
**allora** il sistema registra la prenotazione e mostra una conferma.

### AC2 – Libro non disponibile

**Dato** un libro non disponibile,  
**quando** lo studente seleziona "Prenota",  
**allora** il sistema non consente la prenotazione e mostra un messaggio "Libro non disponibile".

## MODELLO

rende esplicito il comportamento



## INFORMAZIONE ESPLICITA

scenario, regole, vincoli



## CRITERIO VERIFICABILE

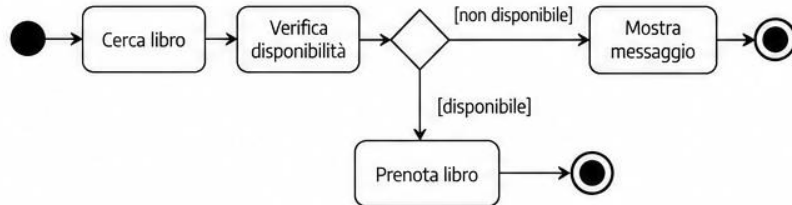
acceptance criteria, test

**Una buona specification non descrive solo cosa vogliamo:**  
rende chiaro come verificare che sia stato implementato correttamente.

## Cosa abbiamo finora?

Modello e informazioni estratte

### Activity Diagram (semplificato)



### Scenario principale

1. Lo studente cerca un libro nel catalogo.
2. Il sistema verifica la disponibilità.
3. Se il libro è disponibile, lo studente lo prenota.
4. Il sistema registra la prenotazione e mostra una conferma.



### Business rule

Un libro non disponibile non può essere prenotato.



### Acceptance criteria (esempi)

#### AC1 – Prenotazione con libro disponibile

Dato un libro disponibile, quando lo studente seleziona “Prenota”, allora il sistema registra la prenotazione e mostra una conferma.

#### AC2 – Libro non disponibile

Dato un libro non disponibile, quando lo studente seleziona “Prenota”, allora il sistema non consente la prenotazione e mostra un messaggio.



## È sufficiente per implementare la feature?

**No. Ci sono ancora molte domande aperte.**



### Chi può prenotare?

- Solo studenti autenticati?
- Utenti sospesi possono prenotare?

Regole di accesso non definite



### Cosa significa “disponibile”?

- Almeno una copia disponibile?
- Si prenota il libro o una copia specifica?

Semantica della disponibilità da chiarire



### Quante prenotazioni?

- Uno studente può prenotare più copie dello stesso libro?
- Esiste un limite di prenotazioni attive?

Limiti e vincoli non definiti



### Cosa succede dopo?

- Quanto dura la prenotazione?
- Quando scade?
- Il libro cambia stato?

Ciclo di vita della prenotazione da definire



### Concorrenza

- Cosa succede se due studenti prenotano contemporaneamente l'ultima copia?

Gestione della concorrenza da specificare



### Errori e casi alternativi

- Cosa succede se la disponibilità cambia durante l'operazione?
- Come gestiamo errori di sistema?

Casi eccezionali da dettagliare



### Feedback all'utente

- Quali informazioni mostriamo dopo la prenotazione?
- Mostriamo la data di scadenza?

Messaggi e contenuti da definire



### Requisiti non funzionali

- Ci sono requisiti di performance?
- Audit log delle prenotazioni?

NFR da verificare nel D1



**Il modello ci aiuta a capire il flusso e alcuni elementi chiave, ma non è sufficiente per implementare la feature senza fare assunzioni.**

Le domande emerse devono essere chiarite (Feature Discovery / Clarification) e poi incluse nella specification.



## Domande emerse dal modello

Esempio: Prenotare un libro (EasyLib)



### Chi può prenotare?

Solo studenti autenticati?  
Altri ruoli?



### Cosa significa “disponibile”?

Almeno una copia?  
Si prenota il libro o una copia specifica?



### Quanto dura la prenotazione?

Esiste una data di scadenza?  
È rinnovabile?



### Quante prenotazioni può avere un utente?

Esiste un limite di prenotazioni attive?



### Cosa succede se l'ultima copia viene prenotata contemporaneamente?

Come gestiamo la concorrenza?



### Quali informazioni mostrare all'utente?

Messaggio di conferma, data di scadenza, prossimi passi?



## Clarification

Discussione con stakeholder



Stakeholder + team  
chiariscono le decisioni  
di dominio, risolvono le  
ambiguità e definiscono  
i requisiti mancanti.

### Fonti di chiarimento

- Committente / stakeholder
- D1 (requisiti, use case, mockup)
- Vincoli organizzativi
- Policy e regolamenti
- Discussione con il team



## Decisioni

Risultato della clarification

Solo studenti autenticati possono prenotare.

DECISO

“Disponibile” = almeno una copia prenotabile. Si prenota una copia specifica.

DECISO

Durata della prenotazione: 7 giorni.  
Rinnovabile una volta.

DECISO

Massimo 3 prenotazioni attive per utente.

DECISO

Prenotazione atomica: se l'ultima copia viene prenotata da un altro utente, la richiesta viene rifiutata.

DECISO

Mostrare messaggio di conferma con data di scadenza e link alla propria area personale.

DECISO



### MODELLO

rappresenta ciò che sappiamo  
e fa emergere ciò che manca



### DOMANDE

ambiguità, informazioni  
mancanti, GAP



### CLARIFICATION

discussione e decisioni  
di dominio



### DECISIONI

risposte esplicite  
o GAP



### SPECIFICATION

requisiti completi,  
coerenti e verificabili



**Il modello non serve solo a rappresentare ciò che sappiamo.  
Serve anche a far emergere ciò che non sappiamo ancora.**

# Feature Specification – Esempio: Prenotare un libro

Una feature può derivare da più elementi del D1

Progetto di esempio

EasyLib

## D1 – Elementi collegati a questa feature

La feature è stata identificata tramite l'analisi congiunta di più elementi del D1.



Feature

**F03**

Prenotare un libro

## Riferimenti D1 (esempi)

- RF3 – Prenotazione libro
- UC2 – Prenotare un libro
- US5 – Come studente voglio prenotare un libro
- AC da D1 (es. disponibilità, conferma prenotazione)
- Mockup – schermata di prenotazione
- User Flow – flusso di prenotazione
- RNF1 – Autenticazione utenti
- RNF3 – Performance
- Altri riferimenti ...



## Obiettivo

Consentire a un utente autorizzato di prenotare una copia disponibile di un libro, secondo le regole definite dal servizio.



## Precondizioni

- Utente autenticato
- Account abilitato alla prenotazione
- Libro presente nel catalogo
- Almeno una copia prenotabile disponibile



## Scenari alternativi / Edge Cases

- ALT1** Libro non disponibile
- ALT2** Prenotazione già esistente per lo stesso libro
- ALT3** Utente non autorizzato
- ALT4** Disponibilità cambiata durante l'operazione
- ALT5** Errore durante la registrazione



## Dati / Stati rilevanti

- Entità coinvolte: Libro, Copia, Prenotazione, Utente
- Stati di prenotazione: attiva, scaduta, cancellata, ritirata
- Attributi principali: dataPrenotazione, dataScadenza



## Requisiti non funzionali applicabili

- RNF1 – Sicurezza: solo utenti autenticati possono prenotare.
- RNF3 – Performance: verifica disponibilità entro 2 secondi.
- Altri RNF: da valutare.



## Attori

- Studente autenticato (principale)
- Altri ruoli: da chiarire (es. docente, personale biblioteca)



## Scenario principale

1. L'utente visualizza il dettaglio del libro.
2. Il sistema mostra la disponibilità (numero di copie prenotabili).
3. L'utente richiede la prenotazione.
4. Il sistema verifica che la prenotazione sia consentita.
5. Il sistema registra la prenotazione.
6. Il sistema mostra la conferma (con data di scadenza).



## Business Rules

- BR1** Una prenotazione può essere creata solo se esiste una copia disponibile.
- BR2** Uno studente non può avere più prenotazioni attive dello stesso libro.
- BR3** La prenotazione scade dopo 7 giorni.
- BR4** Numero massimo di prenotazioni attive per utente: 3.
- BR5** La prenotazione è non trasferibile.



## Acceptance Criteria (esempi)

### AC1 – Prenotazione consentita

**Dato quando allora** un utente autorizzato e un libro prenotabile, l'utente richiede la prenotazione, il sistema registra la prenotazione e mostra la conferma (inclusa la data di scadenza).

### AC2 – Prenotazione non consentita

**Dato quando allora** un libro non prenotabile, l'utente tenta di prenotarlo, il sistema non crea la prenotazione e comunica l'esito (es. "Libro non disponibile").



## GAP / Open Questions

- Altri ruoli autorizzati alla prenotazione?
- Politiche di cancellazione della prenotazione?
- Notifiche (email) alla scadenza?
- Integrazioni con sistemi esterni (es. cataloghi nazionali)?
- Altri GAP emersi durante la modellazione?



**Questa specificazione è il risultato della fase di modellazione e di chiarificazione.**

Integra più elementi del D1, rende esplicite le regole e i criteri verificabili, e mantiene la traceability con i requisiti originari.

# Specification Quality Gate

La Feature Specification è pronta per il Technical Plan?

Progetto di esempio

**EasyLib**



## Checklist di verifica

Verifichiamo che la specification sia completa, coerente e tracciabile.



### Traceability

È chiaro da quali elementi del D1 deriva la feature?  
(RF, UC, US, AC, mockup, user flow, RNF, ...)



### Scope e attori

È chiaro chi usa la feature e con quale obiettivo?



### Scenario principale

Il comportamento principale è completo e non ambiguo?



### Scenari alternativi / Edge cases

Sono stati considerati i casi alternativi e le condizioni di errore rilevanti?



### Business Rules

Le regole di business sono esplicite, complete e non ambigue?



### Dati e stati

Sono definiti i dati, gli stati e le entità necessari al comportamento della feature?



### Acceptance Criteria

Gli acceptance criteria sono verificabili e misurabili?



### NFR applicabili

I requisiti non funzionali rilevanti sono stati identificati e collegati alla feature?



### GAP / Open Questions

I GAP critici per l'implementazione sono risolti?  
I GAP rimanenti sono chiaramente documentati?



### Coerenza

Testo, modelli (use case, diagrammi, ecc.) e criteri sono coerenti tra loro e con gli elementi del D1?



## Domanda chiave

Un AI Coding Agent potrebbe implementare questa feature  
**senza dover inventare requisiti  
o decisioni di business?**



**SÌ**

### Specification Ready

La specification è sufficientemente completa, coerente e verificabile.



Passa al  
**Technical Plan**



**NO**

### Serve ulteriore lavoro

Torna a Feature Discovery,  
Clarification e/o  
alla Feature Specification.



Torna alla fase  
**precedente**



**Il Quality Gate non verifica che la specification sia “perfetta”.**

Verifica che sia sufficientemente completa, coerente, tracciabile e verificabile per guidare il passo successivo.

# Traceability della Feature – Esempio

Manteniamo il collegamento con gli elementi del D1

Progetto di esempio

**EasyLib**

## Elementi del D1 collegati alla feature

Diversi elementi del D1 contribuiscono alla feature "Prenotare un libro".



**RF3**

Prenotazione libro



**RF5**

Disponibilità copie



**UC2**

Prenotare un libro



**US5**

Come studente voglio prenotare un libro



**AC (dal D1)**

Criteri di accettazione sulla prenotazione



**Mockup**

Schermata di prenotazione



**User Flow**

Flusso di prenotazione



**RNF1**

Autenticazione utenti

...

## Feature

Unità funzionale individuata tramite Feature Discovery.

**F03**

Prenotare un libro

La feature raggruppa e sintetizza i requisiti correlati del D1.

## Feature Specification (estratto)

La specification dettaglia il comportamento della feature e mantiene espliciti i riferimenti al D1.



### Riferimenti D1

RF3, RF5, UC2, US5, AC (D1), Mockup, User Flow, RNF1



### Scenario principale (estratto)

1. L'utente visualizza il dettaglio del libro.
2. Il sistema mostra la disponibilità.
3. L'utente richiede la prenotazione.
4. Il sistema registra la prenotazione e mostra la conferma.



### Business Rule (esempio)

**BR1** – Una prenotazione può essere creata solo se esiste una copia disponibile.



### Acceptance Criteria (esempio)

**AC1** – Prenotazione consentita quando il libro è disponibile.  
**AC2** – Messaggio di errore se non disponibile.



### NFR applicabili (esempio)

**RNF1** – Solo utenti autenticati possono prenotare.  
**RNF3** – Verifica disponibilità entro 2 secondi.

...



**Traceability significa mantenere esplicito da quali elementi del D1 deriva la feature e quali parti della specification li raffinano.**

# Modelli nei diversi momenti dello sviluppo

Stesso sistema, scopi diversi, modelli (anche) condivisi

Progetto di esempio  
**EasyLib**

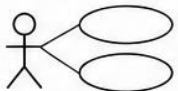
## D1 – Requirements Engineering

Capire e comunicare il problema

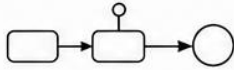
### Domande tipiche

- Quali sono gli attori e i loro obiettivi?
- Quali sono i casi d'uso principali?
- Come funziona il sistema (vista dall'esterno)?
- Quali sono i concetti principali del dominio?
- Quali sono i flussi di attività?

### Esempi di modelli



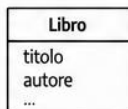
Use Case Diagram



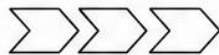
Activity Diagram /  
BPMN



Sequence Diagram  
(alto livello)



Domain Model



User Flow

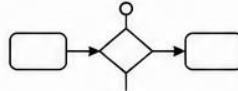
## D2 – Feature Specification

Precisare, disambiguare, rendere verificabile

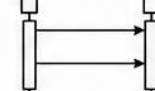
### Domande tipiche

- Quali sono gli scenari principali e alternativi?
- Quali sono le regole di business?
- Quali stati e transizioni sono rilevanti?
- Come si comporta il sistema in dettaglio?
- Come verifichiamo che la feature sia corretta?

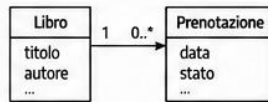
### Esempi di modelli



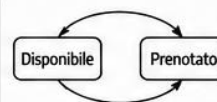
Activity Diagram



Sequence Diagram



Domain Model



State Machine

**IF** una copia è disponibile  
**THEN** la prenotazione può essere creata

Business Rules / OCL

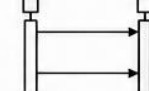
## Technical Design

Progettare la soluzione

### Domande tipiche

- Quali componenti collaborano?
- Come sono strutturate le classi?
- Come vengono gestiti i dati?
- Quali sono le responsabilità dei componenti?
- Come viene distribuito il sistema?

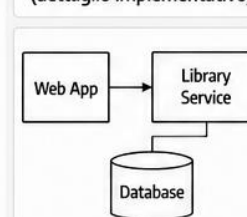
### Esempi di modelli



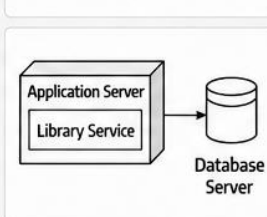
Sequence Diagram  
(dettaglio implementativo)



Class Diagram



Component Diagram



Deployment Diagram



- **Alcuni modelli** possono essere utilizzati in **più fasi**.
- **Cambiano lo scopo, il contenuto e il livello di dettaglio**.
- **Scegliamo il modello in base alla domanda da chiarire**, non in base alla fase.

# F03 – Prenotare un libro

Dalla Feature Specification alle decisioni tecniche nel D3

Progetto di esempio  
**EasyLib**

## Feature Specification (estratto)

Cosa deve succedere?

### Business Rules

- BR1** La prenotazione è possibile solo se esiste almeno una copia disponibile.
- BR2** Uno stesso utente non può avere più di una prenotazione per lo stesso libro.
- BR3** Una prenotazione ha una durata massima di 7 giorni.
- BR4** Ogni utente può avere al massimo 3 prenotazioni attive.

### Acceptance Criteria (estratto)

- AC1** Se il libro è disponibile e l'utente non ha già una prenotazione attiva, la prenotazione viene creata con successo.
- AC2** Se non ci sono copie disponibili, la prenotazione viene rifiutata con un messaggio appropriato.
- AC3** Se l'utente ha già 3 prenotazioni attive, la prenotazione viene rifiutata.

### Requisiti non funzionali (estratto)

- RNF1** Solo utenti autenticati possono prenotare.
- RNF2** Il tempo di risposta deve essere inferiore a 2 secondi.
- RNF3** Il sistema deve gestire richieste concorrenti in modo corretto.
- ...

## Quali decisioni tecniche contribuisce F03?

F03 non ha un Technical Plan autonomo, ma contribuisce a definire le decisioni tecniche del D3 per la soluzione complessiva.

Ad esempio:

- quali componenti implementano la logica di prenotazione;
  - quali dati devono essere gestiti;
  - quali interazioni sono necessarie;
  - quali regole di concorrenza e validazione sono richieste;
  - quali API devono essere esposte;
  - quali requisiti di prestazioni e sicurezza devono essere considerati;
- ...

## Contributo di F03 al Technical Plan (D3)

Esempi di modelli e decisioni tecniche



### Architettura e componenti

- Quali componenti implementano la prenotazione?
- Come si integra con utenti, catalogo e notifiche?



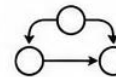
### Interazioni

- Quali sono i flussi principali (es. crea prenotazione, verifica disponibilità, scadenza)?
- Quali sono le interazioni con altri servizi?



### Modello dati

- Quali entità (Prenotazione, Libro, Copia)?
- Quali vincoli sui dati (es. unicità, durata, stato)?



### Stati e transizioni

- Qual è il ciclo di vita di una prenotazione?
- Quali eventi ne modificano lo stato (creazione, scadenza, cancellazione)?



### API

- Quali endpoint sono necessari (es. create, cancel)?
- Quali input, output e codici di errore?



### Altre decisioni tecniche

- Gestione della concorrenza e delle prenotazioni simultanee
  - Strategia di validazione e gestione errori
  - Considerazioni di performance e sicurezza
- ...



**La Feature Specification definisce il comportamento atteso.**

**Le decisioni tecniche per realizzarlo sono parte del D3,** che integra il contributo di tutte le feature in una soluzione coerente.

# Dalle feature al Technical Plan della soluzione

Più feature, una soluzione integrata

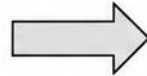
Progetto di esempio  
**EasyLib**

## 1. Feature Specification (WHAT)

Una specifica per ogni feature

|                                          |                                                                                                                                                                                                                                                |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>F01</b><br>Ricerare un libro          |  <ul style="list-style-type: none"><li>• Comportamento</li><li>• Business rules</li><li>• Acceptance criteria</li><li>• Modelli (es. Activity, ...)</li></ul> |
| <b>F02</b><br>Visualizzare disponibilità |  <ul style="list-style-type: none"><li>• Comportamento</li><li>• Business rules</li><li>• Acceptance criteria</li><li>• Modelli (es. Activity, ...)</li></ul> |
| <b>F03</b><br>Prenotare un libro         |  <ul style="list-style-type: none"><li>• Comportamento</li><li>• Business rules</li><li>• Acceptance criteria</li><li>• Modelli (es. Activity, ...)</li></ul> |
| <b>F04</b><br>Ricevere una notifica      |  <ul style="list-style-type: none"><li>• Comportamento</li><li>• Business rules</li><li>• Acceptance criteria</li><li>• Modelli (es. Activity, ...)</li></ul> |

### Specification Quality Gate



Tutte le feature sono sufficientemente specificate e pronte per la progettazione della soluzione.

Ogni feature viene specificata in modo indipendente, con i modelli necessari a descrivere il comportamento e le regole di business.

## 2. Design Integration (HOW)

Integriamo le esigenze delle feature

### Input

- Feature Specification (F01, F02, F03, F04, ...)
- Requisiti non funzionali
- Vincoli tecnologici e di contesto
- Goal architetturali (es. modularità, scalabilità, ...)



### Technical Plan della soluzione

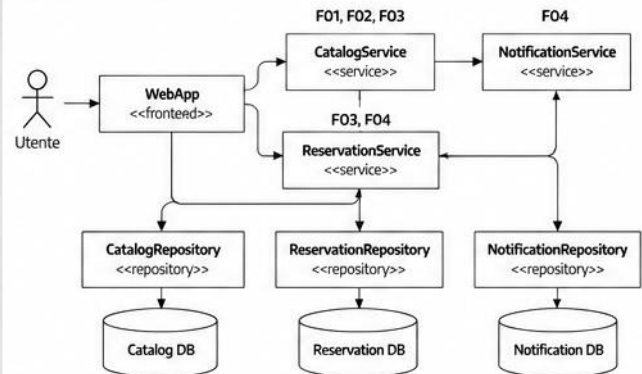
- Architettura complessiva
- Modelli condivisi (componenti, dati, ...)
- Decisioni tecniche integrate
- Mappatura feature → componenti
- Piano di implementazione (tasks)

Il Technical Plan non è la somma di piani per singole feature, ma un progetto coerente e integrato dell'intera soluzione.

## 3. Risultato: una soluzione condivisa

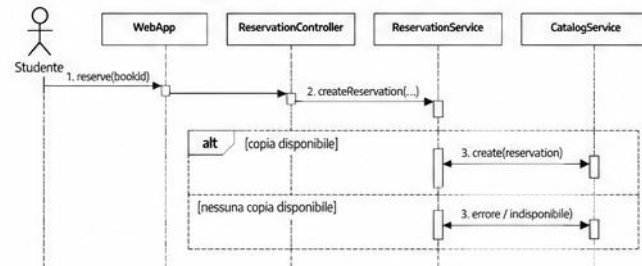
Le feature convergono sugli stessi componenti

### Component Diagram – EasyLib



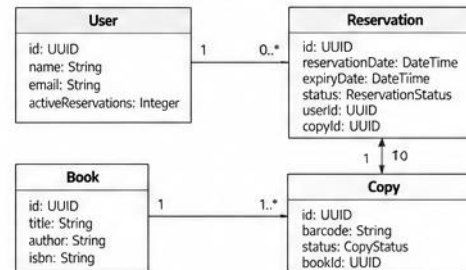
Ogni componente può supportare più feature.  
La mappatura feature → componenti garantisce la traceability e aiuta a prendere decisioni coerenti.

### Esempio: Sequence Diagram (F03 – Prenotare un libro)



Il Sequence Diagram descrive uno scenario specifico di una feature, utilizzando i componenti della soluzione condivisa.

### Esempio: Class/Data Model (condiviso)



Il modello dei dati integra i concetti richiesti da più feature (F01, F02, F03, F04).

### Esempio: State Machine (F03 – Reservation)



La State Machine descrive il ciclo di vita di una prenotazione e supporta le decisioni implementative condivise.



**Specificare può essere feature-oriented.**  
**Progettare richiede una visione integrata della soluzione.**

I modelli non sono deliverable isolati: accompagnano le feature e convergono in un Technical Plan coerente, che guida l'implementazione dell'intero sistema.

# Technical Plan: modelli condivisi tra più feature

Diverse prospettive, diversi scope

Progetto di esempio  
**EasyLib**

## Modelli feature-specific

Descrivono il comportamento di una singola feature

### Utili per:

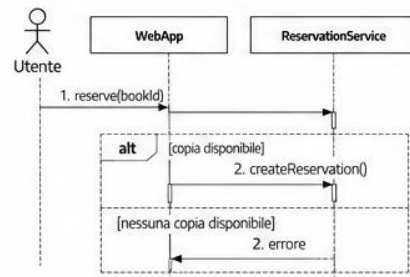
- descrivere il flusso o il comportamento di una feature;
- rappresentare casi specifici e scenari alternativi;
- validare la logica della feature con gli stakeholder;
- guidare la progettazione dettagliata e i test.

### Esempi (EasyLib)

#### Activity Diagram (F01 – Ricercare un libro)



#### Sequence Diagram (F03 – Prenotare un libro)



#### State Machine (Reservation)



### Altri esempi

- Sequence Diagram (F04 – Notifica disponibilità)
- Activity Diagram (F02 – Visualizzare disponibilità)
- Use Case / scenari specifici
- Modelli di dominio parziali (se utili per la feature)

Questi modelli aiutano a capire cosa fa la feature.

Possono essere più semplici e focalizzati sullo scenario specifico.

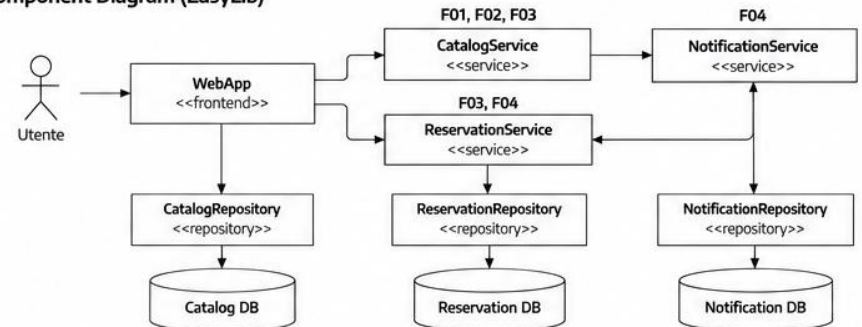
## Modelli condivisi (solution-wide)

Integrano le esigenze di più feature e descrivono la soluzione nel suo insieme

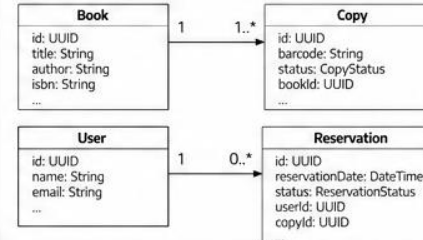
### Utili per:

- definire l'architettura complessiva della soluzione;
- integrare i requisiti tecnici di più feature;
- progettare i componenti, i dati e le loro relazioni;
- prendere decisioni coerenti su tecnologia, deployment e infrastruttura;
- garantire la traceability tra feature e componenti.

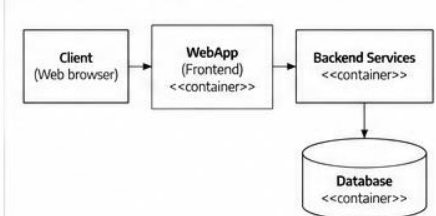
### Component Diagram (EasyLib)



### Class/Data Model (condiviso)



### Deployment Diagram (semplificato)



Questi modelli descrivono come è costruita la soluzione.

Integrano decisioni provenienti da più feature e vengono evoluti man mano che nuove feature vengono aggiunte.



**Non tutti i modelli hanno lo stesso scope.**

Alcuni descrivono una feature, altri integrano più feature nella soluzione tecnica.

**Usa i modelli che servono.**

L'obiettivo è ridurre ambiguità e rendere esplicite le decisioni rilevanti.

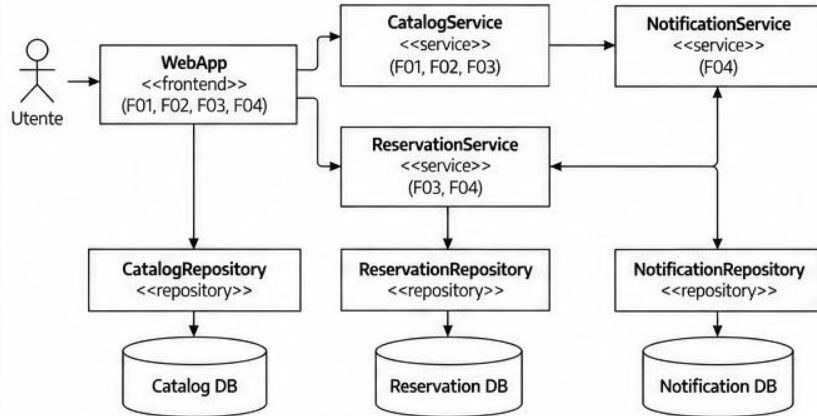
# Feature-to-Component Mapping

Più feature, gli stessi componenti

Progetto di esempio  
**EasyLib**

## Component Diagram (EasyLib)

Componenti della soluzione e feature supportate



## Mappatura Feature → Componenti

Quali componenti sono coinvolti in ciascuna feature

| Feature                          | WebApp (Frontend) | CatalogService | ReservationService | NotificationService | Database (Repositories) |
|----------------------------------|-------------------|----------------|--------------------|---------------------|-------------------------|
| F01 – Ricercare un libro         | ✓                 | ✓              |                    |                     | ✓                       |
| F02 – Visualizzare disponibilità | ✓                 | ✓              |                    |                     | ✓                       |
| F03 – Prenotare un libro         | ✓                 | ✓              | ✓                  |                     | ✓                       |
| F04 – Ricevere una notifica      | ✓                 | ✓              | ✓                  | ✓                   | ✓                       |

### Lettura della mappatura

- Più feature possono richiedere gli stessi componenti.
- Ogni componente può supportare più feature.
- La mappatura aiuta a mantenere la traceability e a capire l'impatto di cambiamenti.

## Componenti e feature supportate

| WebApp (Frontend)       | F01 | F02 | F03 | F04 | Interfaccia utente, gestione richieste, visualizzazione risultati.         |
|-------------------------|-----|-----|-----|-----|----------------------------------------------------------------------------|
| CatalogService          | F01 | F02 | F03 |     | Ricerca libri, informazioni sul catalogo, disponibilità.                   |
| ReservationService      | F03 | F04 |     |     | Gestione prenotazioni, regole di business, stati della prenotazione.       |
| NotificationService     | F04 |     |     |     | Invio notifiche, gestione preferenze.                                      |
| Database (Repositories) | F01 | F02 | F03 | F04 | Persistenza dei dati (libri, copie, prenotazioni, notifiche, utenti, ...). |

## Esempi di scenari condivisi

- F01 e F02 utilizzano entrambi CatalogService e accedono agli stessi dati del catalogo.
- F03 utilizza CatalogService (per verificare la disponibilità) e ReservationService (per creare la prenotazione).
- F04 utilizza ReservationService (per monitorare lo stato) e NotificationService (per inviare la notifica).

### Perché è importante?

- Aiuta a progettare una soluzione coerente e senza duplicazioni.
- Rende esplicite le dipendenze tra feature e componenti.
- Supporta l'analisi dell'impatto quando una feature o un componente cambia.
- Facilita la pianificazione delle attività e dei test.



**Il Technical Plan non appartiene a una singola feature:**  
integra decisioni condivise e mantiene visibile quali feature richiedono ciascun componente.

Quando cambia una feature, la mappatura ci aiuta a capire quali parti della soluzione potrebbero essere impattate.

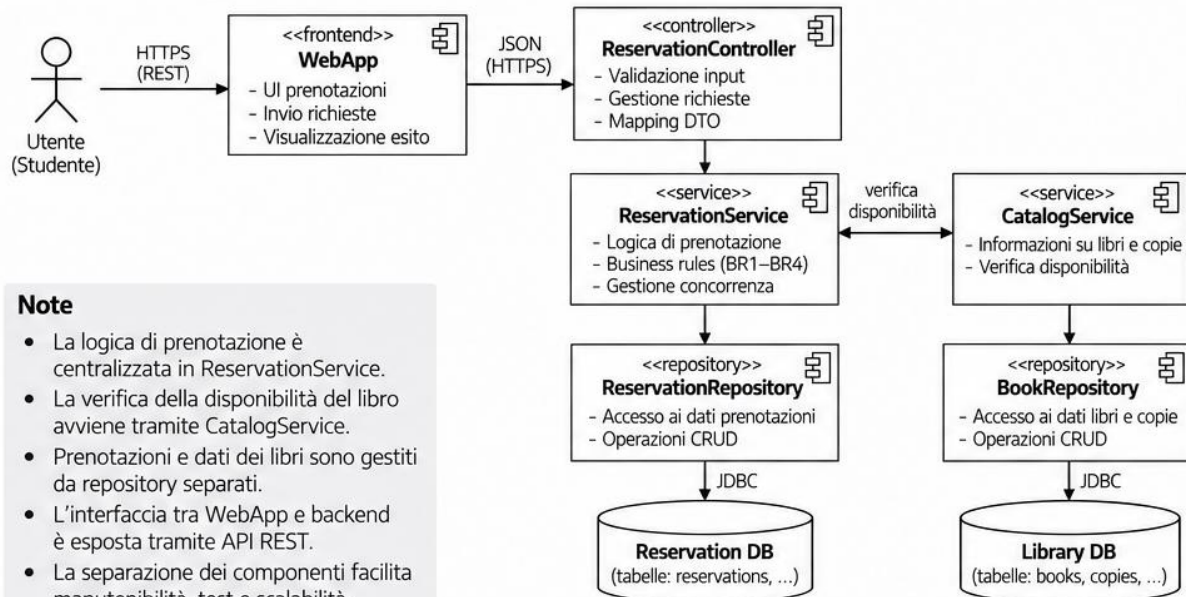
# F03 – Prenotare un libro

## Technical Plan – Component Diagram

Progetto di esempio  
**EasyLib**

Vista d'insieme dei componenti e delle loro interazioni nella gestione delle prenotazioni

### Component Diagram



#### Note

- La logica di prenotazione è centralizzata in ReservationService.
- La verifica della disponibilità del libro avviene tramite CatalogService.
- Prenotazioni e dati dei libri sono gestiti da repository separati.
- L'interfaccia tra WebApp e backend è esposta tramite API REST.
- La separazione dei componenti facilita manutenibilità, test e scalabilità.

### Responsabilità principali

- WebApp**  
Interfaccia utente per la prenotazione.
- ReservationController**  
Gestisce le richieste HTTP e valida gli input.
- ReservationService**  
Implementa le business rules (BR1-BR4).
- CatalogService**  
Fornisce informazioni su libri e copie (verifica disponibilità).
- ReservationRepository**  
Gestisce la persistenza delle prenotazioni.
- BookRepository**  
Gestisce libri e copie.
- Database**  
Memorizza dati di prenotazioni, libri e copie.

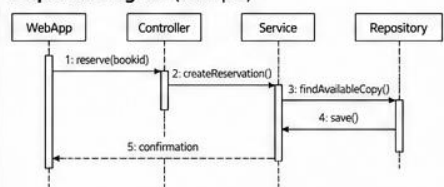
### Collegamento con la Specification

- BR1 (copia disponibile) → CatalogService
- BR2 (niente doppia prenotazione) → ReservationService
- BR3 (scadenza 7 giorni) → ReservationService
- BR4 (max 3 prenotazioni) → ReservationService
- RNF1 (autenticazione) → WebApp / Controller
- RNF2 (tempo di risposta) → Scelte architetturali
- RNF3 (concorrenza) → Repository / DB

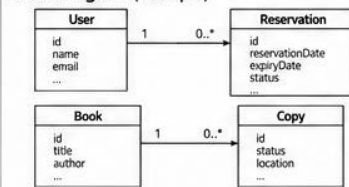
### Altri diagrammi utili per questa feature

Il Component Diagram mostra la struttura. Altri modelli possono essere utilizzati per approfondire aspetti specifici del Technical Plan.

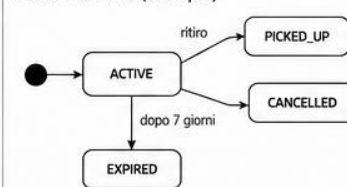
#### Sequence Diagram (esempio)



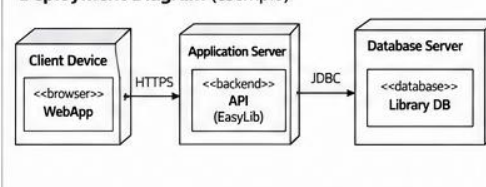
#### Class Diagram (esempio)



#### State Machine (esempio)



#### Deployment Diagram (esempio)



Non servono tutti i diagrammi: scegliamo i modelli che aiutano a rendere esplicite le decisioni tecniche rilevanti per la feature.

# F03 – Prenotare un libro

## Technical Plan – Sequence Diagram

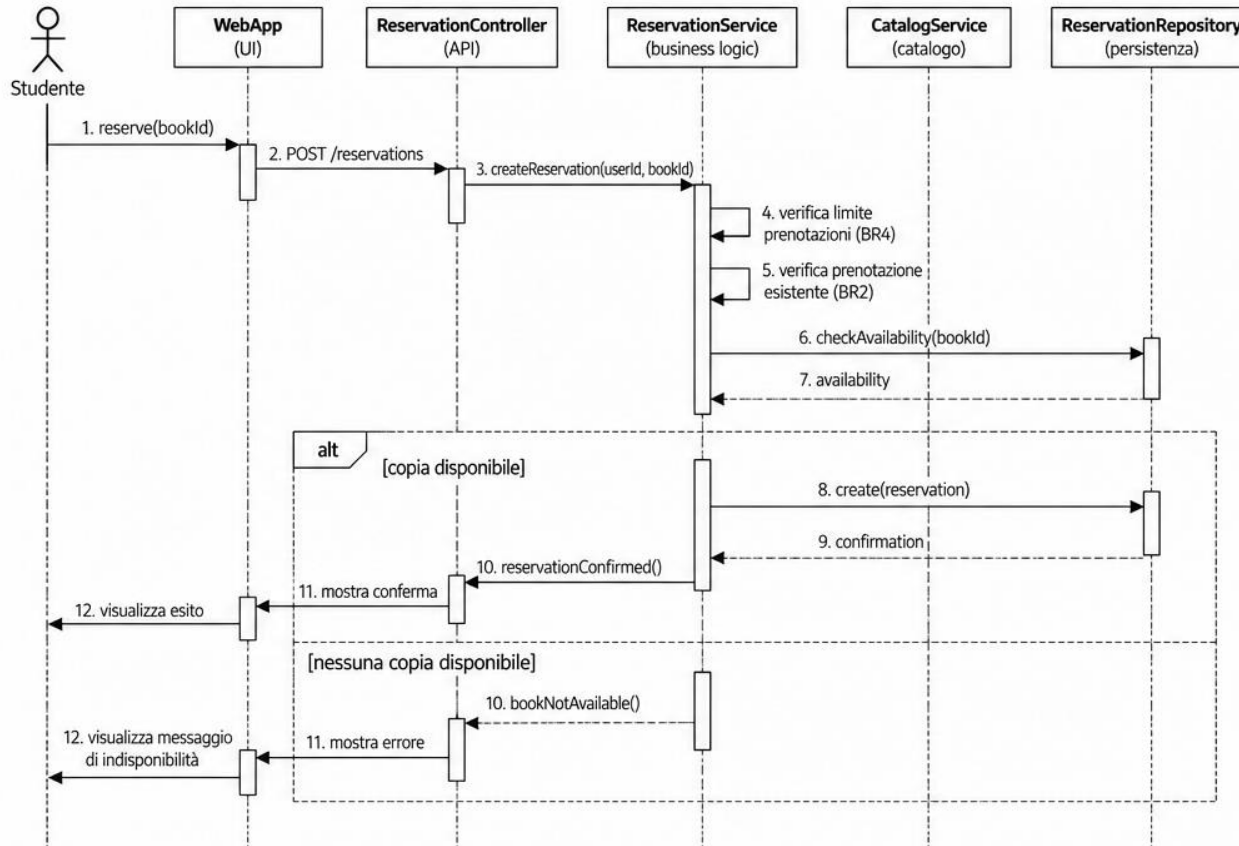
Progetto di esempio

**EasyLib**

Come collaborano i componenti per realizzare una prenotazione

### Sequence Diagram

Flusso principale e gestione del caso alternativo



### Traceability con la Specification

- BR1 → passo 6–7  
(verifica disponibilità della copia)
- BR2 → passo 5  
(verifica prenotazione già esistente)
- BR4 → passo 4  
(verifica limite prenotazioni attive)
- AC1 → passo 8–12 (flusso principale, prenotazione confermata)
- AC2 → passo 10–12 (flusso alternativo, prenotazione non consentita)

### Note sul diagramma

- Mostra la collaborazione tra i componenti identificati nel Component Diagram.
- Le verifiche delle business rules sono realizzate nel ReservationService.
- La disponibilità del libro viene verificata tramite CatalogService.
- La persistenza della prenotazione è gestita da ReservationRepository.
- Il diagramma mostra sia il flusso di successo sia un caso alternativo (nessuna copia disponibile).
- Altri scenari (es. utente non autenticato, prenotazioni concorrenti) possono essere modellati in diagrammi aggiuntivi.

### Possibili estensioni

- Gestione di errori specifici (es. utente non autorizzato).
- Timeout e retry nelle chiamate ai servizi.
- Gestione della concorrenza (es. lock, transazioni).
- Invio di notifiche all'utente (es. email di conferma).



**Il Sequence Diagram traduce il comportamento della specification in collaborazioni tra i componenti della soluzione.**

Permette di vedere dove e come vengono realizzate le decisioni della specification nella soluzione tecnica.

# F03 – Prenotare un libro

## Technical Plan – Struttura dei dati e ciclo di vita

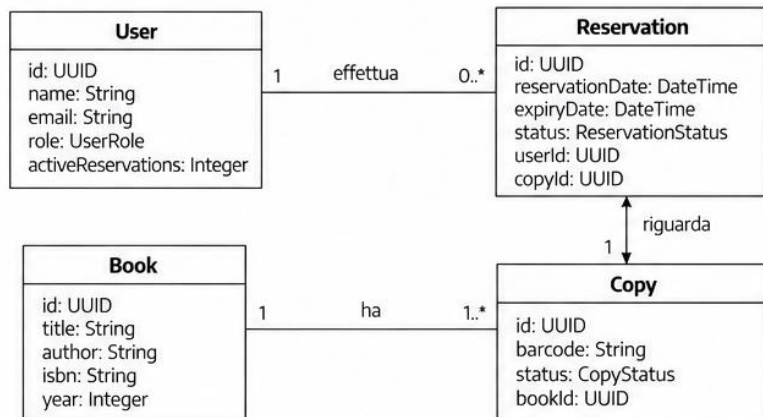
Progetto di esempio

**EasyLib**

### Modello dei dati principali e stati della prenotazione

#### Class Diagram

Principali entità e relazioni



#### Note sul modello

- Un libro (Book) può avere una o più copie (Copy).
- Una prenotazione (Reservation) riguarda una specifica copia di un libro.
- Un utente (User) può avere più prenotazioni, ma al massimo 3 attive (BR4).
- Sono mostrati solo gli attributi principali.
- Tipi ed eventuali vincoli possono essere dettagliati nel design del database.

#### Enumerazioni (esempio)

| ReservationStatus | CopyStatus  |
|-------------------|-------------|
| CREATED           | AVAILABLE   |
| ACTIVE            | RESERVED    |
| PICKED_UP         | ON_LOAN     |
| CANCELLED         | UNAVAILABLE |
| EXPIRED           |             |

#### State Machine – Reservation

Ciclo di vita di una prenotazione



#### Note sullo stato

- Una prenotazione viene creata e passa in stato ACTIVE quando la richiesta è accettata e la copia è riservata.
- La prenotazione rimane ACTIVE per un massimo di 7 giorni (BR3), dopo di che passa a EXPIRED.
- L'utente o lo staff possono cancellare la prenotazione in qualsiasi momento (stato CANCELLED).
- Quando il libro viene ritirato, la prenotazione passa in stato PICKED\_UP.

#### Eventi e transizioni (esempio)

| Evento        | Da      | A         | Condizione / Note              |
|---------------|---------|-----------|--------------------------------|
| conferma      | CREATED | ACTIVE    | Copia disponibile (BR1)        |
| ritiro        | ACTIVE  | PICKED_UP | Ritiro del libro (dall'utente) |
| cancellazione | ACTIVE  | CANCELLED | Richiesta utente o staff       |
| scadenza      | ACTIVE  | EXPIRED   | Dopo 7 giorni (BR3)            |

#### Collegamento con la Specification

- **BR1** – La prenotazione è possibile solo se esiste almeno una copia disponibile → Copy.status = AVAILABLE
- **BR3** – Una prenotazione ha una durata massima di 7 giorni → Reservation.expiryDate e transizione a EXPIRED
- **BR4** – Ogni utente può avere al massimo 3 prenotazioni attive → User.activeReservations (e controlli nel service)
- **AC1** – Prenotazione creata con successo → stato ACTIVE
- **AC2** – Prenotazione rifiutata se non ci sono copie disponibili → nessuna transizione, errore nel flusso

#### Possibili estensioni

- Gestione di stati aggiuntivi (es. ON\_HOLD, REJECTED).
- Storico degli stati della prenotazione.
- Gestione delle notifiche (es. promemoria di ritiro).
- Attributi aggiuntivi per Book e Copy (es. categoria, posizione, sede).
- Vincoli e indici sul database (es. unicità, performance).



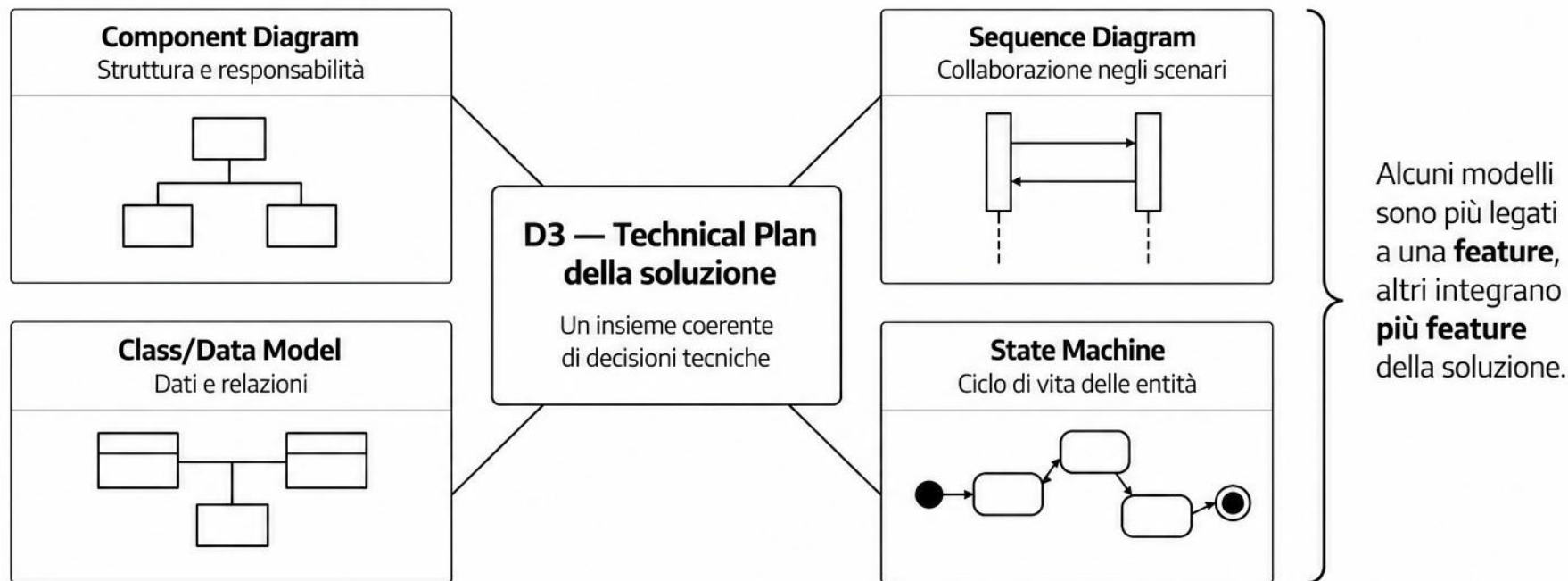
**Il Class/Data Model chiarisce quali informazioni devono essere rappresentate.**  
**La State Machine chiarisce come cambia lo stato della prenotazione nel tempo.**

Insieme, questi modelli supportano la progettazione di una soluzione coerente con la specification.

# D3 – Technical Plan della soluzione

Modelli diversi, un'unica visione coerente

Progetto di esempio  
**EasyLib**



**D2 — Feature Specifications**

Design  
Integration

Technical  
Models

**D3 — Technical Plan della soluzione**



Il D3 integra le Feature Specification del D2 in una soluzione tecnica coerente, supportata dai modelli necessari.

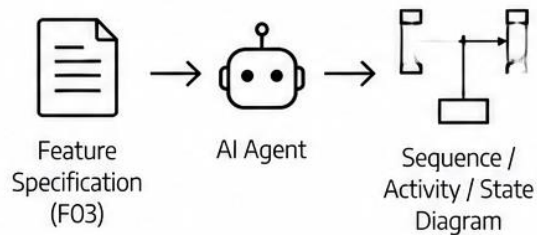
# Modelli e AI Coding Agents

Come i modelli supportano lo sviluppo Spec-Driven

Progetto di esempio  
**EasyLib**

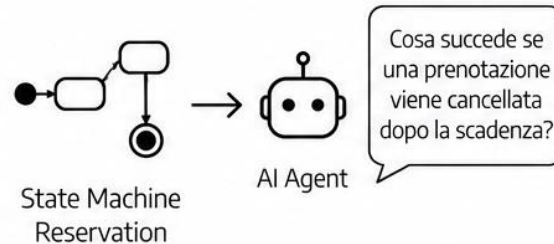
## 1 Specification → Model Generazione di modelli

L'AI può aiutarci a generare un modello a partire dalla specification.



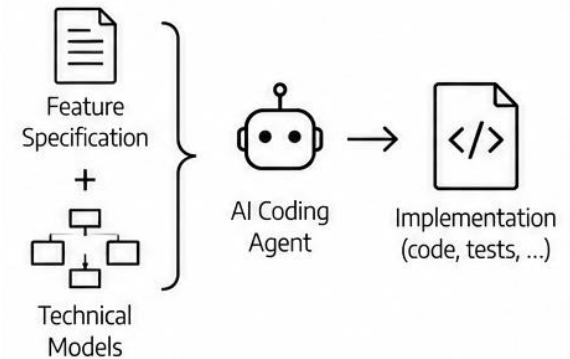
## 2 Model → Analysis Verifica e chiarimenti

L'AI può analizzare un modello per individuare ambiguità, inconsistenze e possibili GAP.



## 3 Specification + Models → Implementation Contesto per l'implementazione

Specification e modelli forniscono insieme contesto all'AI Coding Agent per generare l'implementazione.



### Human Gate

L'AI può generare e interpretare modelli, ma non dobbiamo assumere che il modello generato sia corretto.

**AI genera / modifica → persona verifica → modello accettato.**